IB Comp Sci Notes

Kenny Lin / BusyPizza

October 31, 2026

Abstract

This guide / note is dedicated to the love of my life: computer science. Wish y'all have a great time studying this subject! (第一页用了纯黑背景其实是为了配合下面的插图，后面是正常的白底黑字)
Notes to Mr. Sanchez: this note is for my own use (and I might share this with classmates later this year once it reaches high level of completion), therefore you might see Chinese in it. Simply ignore it.



Contents

A Concepts of Computer Science **4**
A1 Computer fundamentals . . . 4
A1.1 Computer hardware and operation . . . 4
A1.1.1 Function and interaction of the main CPU components . . . 4
A1.1.2 Role of GPU . . . 5
A1.1.3 Difference between CPU and GPU . . . 5
A1.1.4 Purposes of different types of primary memory . . . 6
A1.1.5 FDE cycle . . . 7
A1.1.6 Pipelining in multi-core architectures . . . 9
A1.1.7 Internal and external types of secondary memory storage . . . 9
A1.1.8 Compression . . . 10
A1.1.9 Type of services in cloud implementation . . . 10
A1.2 Data representation and computer logic . . . 11
A1.2.1 Principal methods of representing data . . . 11
A1.2.2 How binary is used to store data . . . 12
A1.2.3 Purpose and use of logic gates . . . 16
A1.2.4 Constructing and analysing truth tables . . . 16
A1.2.5 Constructing logic diagrams . . . 20
A1.3 Operating systems and control systems . . . 20
A1.3.1 The role of operating systems . . . 20
A1.3.2 Functions of an operating system . . . 20
A1.3.3 Approaches for scheduling . . . 26
A1.3.4 Interrupt handling and polling . . . 28
A1.3.5 The role of the operating system in managing multitasking and resource allocation . . . 29
A1.3.6 The use of control system components . . . 31
A1.3.7 The uses of control systems . . . 33
A1.4 Translation . . . 34
A1.4.1 Translation processes of interpreters and compilers . . . 34
A1.4.2 Evaluating the translation processes . . . 37
A1.4.3 Example scenarios . . . 38
A2 Networks . . . 39
A2.1 Network fundamentals . . . 39
A2.2 Network architecture . . . 39
A2.3 Data transmission . . . 39
A2.4 Network security . . . 39
A3 Databases . . . 39
A3.1 Database fundamentals . . . 39
A3.2 Database design . . . 39
A3.3 Database programming . . . 39
A3.4 Alternative databases and data warehouses . . . 39
A4 Machine learning . . . 39
A4.1 Machine learning fundamentals . . . 39
A4.2 Data preprocessing . . . 39
A4.3 Machine learning approaches . . . 39
A4.4 Ethical considerations . . . 39

B Computational Thinking and Problem Solving **40**
B1 Computational thinking . . . 40
B1.1 Approaches to computational thinking . . . 40
B1.1.1 Problem specification . . . 40
B1.1.2 Fundamental concepts of computational thinking . . . 42
B1.1.3 Applying computational thinking to solve problems . . . 42
B1.1.4 Tracing flowcharts for programming algorithms . . . 44
B2 Programming . . . 47
B2.1 Programming fundamentals . . . 47
B2.2 Programming constructs . . . 47
B2.3 Data structures . . . 47
B2.4 Programming algorithms . . . 47
B2.5 File processing . . . 47
B3 Object-oriented programming (OOP) . . . 47
B3.1 Fundamentals of OOP for a single class . . . 47
B3.2 Fundamentals of OOP for multiple classes . . . 47
B4 Abstract data types (ADTs) . . . 47
B4.1 Fundamentals of abstract data types . . . 47

C Appendices **48**
C0.1 My favourite operating system: Arch Linux (+ Hyprland) . . . 48
C0.1.1 What it is . . . 48
C0.1.2 Why it is my favourite . . . 48

A Concepts of Computer Science

A1 Computer fundamentals

A1.1 Computer hardware and operation

A1.1.1 Function and interaction of the main CPU components

The CPU is the critical component that carries out the majority of the processing tasks in a computer system. Consists of:

- Control Unit (CU): Directs the operations of the processor. Responsible for the FDE cycle (managing all three operations) + directing the computer's memory, ALU & I/O devices to carry out the program instructions.
- Arithmetic Logic Unit (ALU): Performs arithmetic and logical operations.
- Registers: Small, fast storage locations within the CPU that hold data and instructions temporarily during processing.
  - Program Counter (PC): Holds the address of the next instruction to be executed.
  - Memory Address Register (MAR): Holds the address of the memory location that is currently being fetched.
  - Memory Data Register (MDR): Holds the data that is being transferred to or from memory.
  - Accumulator (ACC): Holds the result of arithmetic and logical operations performed by the ALU.
  - Instruction Register (IR): Holds the current instruction being executed.
- Buses: Communication pathways that transfer data between the CPU, memory, storage, and I/O devices.
  - Data Bus: Transfers data between the CPU, memory, and I/O devices. Bidirectional.
  - Address Bus: Transfers memory addresses from the CPU to memory. Unidirectional.
  - Control Bus: Transfers control signals from the CPU to other components. Bidirectional.

Core: the processing unit of the CPU. More cores generally means better performance, as multiple cores can handle multiple tasks simultaneously. However, the performance gain depends on the software's ability to utilize multiple cores effectively.
Single core processors work well for naturally sequential tasks, while multi-core processors excel in parallel processing tasks.
Cache: a small, fast memory located close to the CPU that stores frequently accessed data and instructions. It helps reduce the time it takes for the CPU to access data from main memory. (L1, L2, L3)
Co-processors: specialized processors designed to support the main CPU. Tasks from the CPU are offloaded to the co-processor so that they could run in parallel, which can perform them more efficiently. Examples include DSPs, GPUs, and AI accelerators.
Von Neumann vs Harvard architecture: Von Neumann architecture uses a single memory space for both instructions and data, while Harvard architecture uses separate memory spaces for instructions and data. Harvard architecture can provide better performance for certain applications, as it allows simultaneous access to both instructions and data. However, it can be more complex to implement and may require more hardware resources.

A1.1.2 Role of GPU

Used to accelerate the rendering of images, videos, and animations. It is particularly effective for tasks that can be parallelized, such as graphics processing and machine learning.

- GPUs have a large number of cores that can handle multiple tasks simultaneously, making them well-suited for tasks that require high computational power.
- They also handle the application of shaders and textures to 3D models, which includes lighting, shading and texture mapping.
- GPUs assist in decoding and encoding video streams, which is essential for video playback, video editing, and streaming applications.
- Due to the ability to perform parallel processing + matrix and vector operations, GPUs are widely used in machine learning and deep learning applications, where they can accelerate the training and inference of neural networks (esp. for deep learning models).
- GPUs also significantly outperformed CPUs in solving cryptographic puzzles, which is why they are widely used in cryptocurrency mining.

Accelerators: separate main circuit board with the processor, memory, power management, etc., but they can not operate by themselves. They are always part of a system (host) in which the CPUs run the operating systems and control the programs execution. (GPU can be one example as well)

A1.1.3 Difference between CPU and GPU

Heterogeneous programming: CPU acts as a main processor, controlling the execution workflow. The host makes all calls, allocates the memory, and handles the memory transfers between CPU and GPU. GPUs run tens of thousands of threads simultaneously on thousands of cores and does not do much of the data management. The device code is executed by doing calls to functions (kernels) written specifically to take advantage of the GPU.

Overview: The CPU is great for handling various jobs, but the GPU is better for doing the same job many times on a lot of data at once.
Usage scenarios:

- CPU: Used for general-purpose tasks that require flexibility, quick decisions, and coordination—such as running an operating system, web browsing, word processing, and managing program instructions.
- GPU: Used for tasks involving many similar calculations that can be carried out simultaneously / in parallel, such as rendering graphics in video games, training AI models, machine learning, video editing, and large simulations.

Processor	Processing	Architecture	Functionality
CPU	It is a **general** purpose processor, capable of handling many different tasks. It executes the instructions of computer programs, involving operations such as arithmetic, logic and controlling input / output (I/O) operations, as directed by the operating system.	CPUs generally have fewer cores. Each core is very versatile, making it capable of handling complex computations that require sequential processing.	Allows the user to switch between multiple tasks and applications. This makes it ideal for running the operating system and general software applications.
GPU	It is a **specialized** processor, with a focus on handling graphics, **rendering** images, video and animations.	Composed of hundreds or thousands of small cores that are well-suited for tasks that can be run in parallel. While each core is not as powerful as a standard CPU core, the high number of cores allows them to perform a large number of calculations simultaneously, making them perfect for graphical processing.	Suited for tasks that require simultaneous processing of large blocks of data, such as rendering images, video processing and deep learning applications.

In case of collaborative efforts to video games operation:

- The CPU handles the game' s core logic, including rules, physical calculations and AI behaviour. It processes the inputs from the player.
- The GPU renders the game' s visuals. It processes vertex and pixel data to draw images on to the screen, including 3D objects, textures and effects such as lighting and shadows.

A1.1.4 Purposes of different types of primary memory

It directly supports the execution of programs by providing the CPU with fast and flexible access to instructions and data. However, not all primary memory is equal: different types serve distinct purposes, trade-offs, and performance characteristics.

- RAM, Random Access Memory: holds instructions and data for programs that are currently running. Volatile (loses content when powered off) Directly accessible from CPU. Two types of RAM:
  - DRAM: capacitors + transistors (needs to be refreshed)
  - SRAM: flip flops to store data
- ROM, Read-only memory: used for storing instructions that are very rarely modified. (e.g. BIOS: initialize and test the system hardware components on startup, and to load the operating system (OS) software from the secondary memory storage into the RAM)
- Cache (Located within CPU): small, but provides high-speed access to the CPU compared to the RAM. It acts as a buffer between the CPU and the slower RAM, storing frequently used data and instructions. Caches are managed by the cache controller, which uses algorithms like Least Recently Used (LRU) to determine which data to keep or evict.

  - L1: located directly on the CPU, making it the fastest type of cache. However, it is also the smallest, often only a few kilobytes in size (32KB to 128KB per core). Each CPU core usually has its own L1 cache:
    * L1i to store instructions
    * L1d to store data
  - L2: either be on the CPU, or located very near the CPU. L2 cache is larger than L1 and can be up to several megabytes in size (256KB to 2MB per core), but slighly slower than L1 cache.
  - L3: located the furthest from the CPU chip. L3 cache may be shared on multiple-core CPUs. It is the largest of the three, and can be up to tens of megabytes in size (2MB to 64MB shared across all cores). It's also the slowest of all three.

- Registers: fastest and smallest type of memory in a computer system. Located directly inside the CPU, they provide temporary storage for data and instructions that the processor is actively using.
  - General-purpose: used to store operands, intermediate results, memory addresses, or anything else the executing program needs.
  - Special-purpose: used by the CPU control unit to manage and coordinate instruction execution.

Memory Hierarchy: Register ⇒ L1 ⇒ L2 ⇒ L3 ⇒ RAM ⇒ Secondary storage.

Cache hit: data being fetched is located within the cache, reduces retrieval time of data
Cache miss: data being fetched is only located within RAM / secondary storage.
Types of cache miss:

- Compulsory (cold): First-time access to data; it is unavoidable as it is inherent in the first reference to the data.
- Capacity: Cache too small to hold all data.
- Conflict: Multiple addresses map to the same cache line.

The percentage of hit rate determines the efficiency and effectiveness of the cache. Systems with a larger cache size will generally perform better, as well as systems with more intelligent prefetching techniques that can predict which data will be needed soon and load it into cache ahead of time.

A1.1.5 FDE cycle

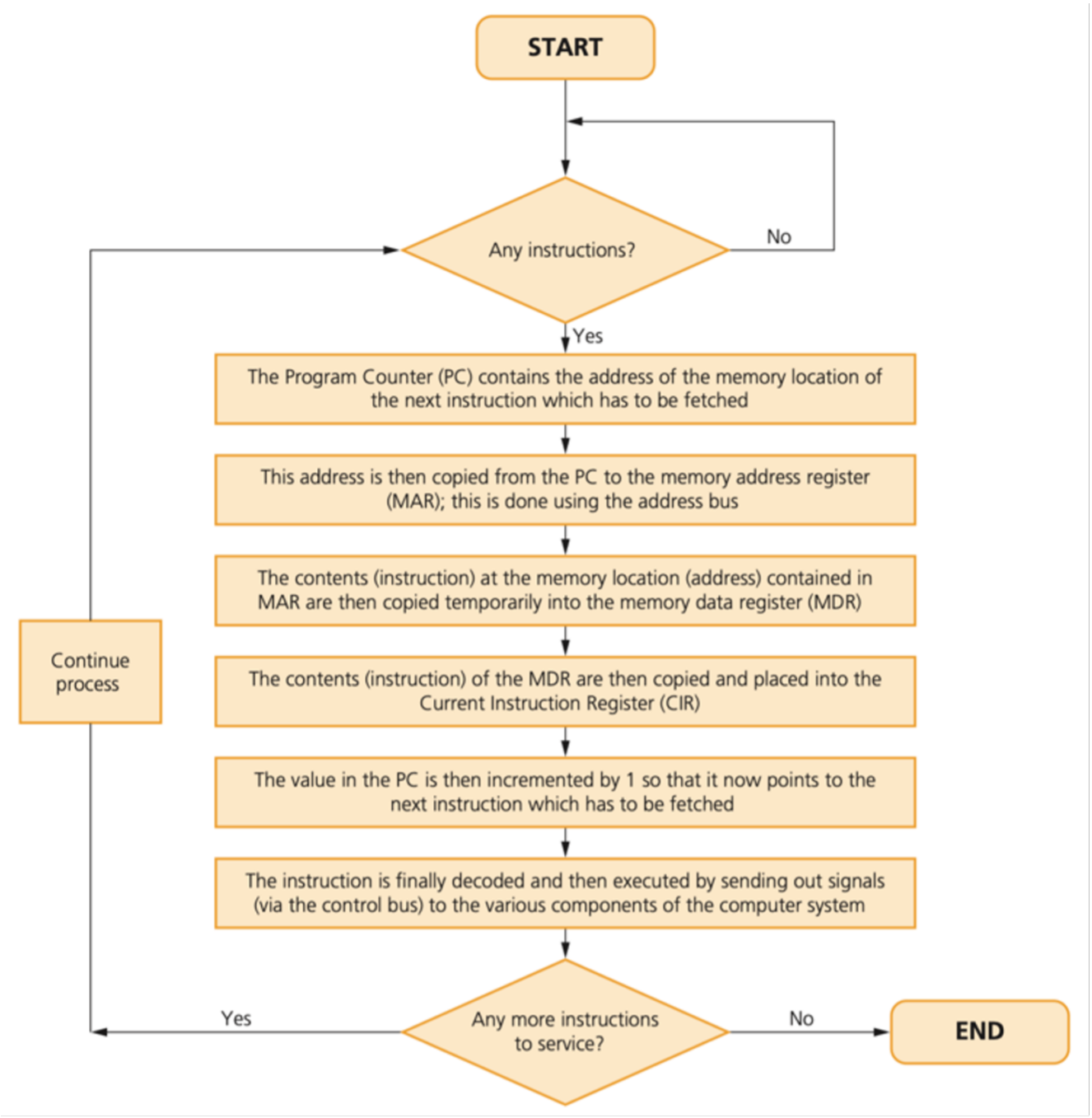
1. Fetch: The CPU fetches an instruction from the memory.
   - The Program Counter (PC) holds the address of the next instruction to be executed.
   - This address is copied into the Memory Address Register (MAR).
   - The address is sent to main memory through the address bus.
   - The instruction stored at that memory location is sent back to the CPU over the data bus and temporarily held in the Memory Data Register (MDR).
   - The instruction in the MDR is then transferred to the Current Instruction Register (CIR), where it awaits decoding.
   - Finally, the PC is incremented so it points to the address of the next instruction.
2. Decode: The CPU interprets the instruction and prepares the necessary operations to execute it.
   - The CU examines the opcode (the part of the instruction that specifies the operation) and determines what the CPU must do.

- If the instruction includes operands (addresses or data values), the CU prepares to retrieve them —possibly using the MAR and MDR again for memory access.
- Internal control signals are generated to direct data movement and select the appropriate ALU operation, register sources, or memory accesses.

3. Execute: The CPU performs the actions required by the instruction.

- Arithmetic or logic operations: Data from registers are processed in the ALU, with results stored in the Accumulator (ACC) or another register.
- Memory operations: The MAR and MDR are used again if data must be read from or written to RAM.
- Control flow operations: The Program Counter (PC) may be updated.

说老实话，这段 IB 教科书上写的内容还不如 IG 时期的教科书，所以我就把去年的拿过来了：



Machine Language: lowest level of programming language that directly corresponds to the instructions executed by a computer's hardware. (In simple terms: binary.)

A1.1.6 Pipelining in multi-core architectures

A1.1.7 Internal and external types of secondary memory storage

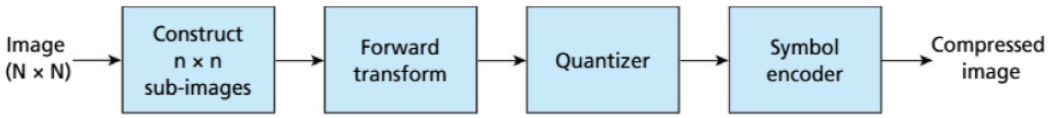
- Internal Storage
  - HDD: utilize a spinning magnetic disk to read / write data. They are suitable for storing large volumes of data, such as media files, backups, and documents, where speed is not so critical.
  - SSD: use flash memory to store data, offering high-speed data access and durability. This makes them very popular in portable devices such as laptops and tablets.
  - eMMC: a type of flash storage that utilizes NAND flash memory. They are soldered directly on to the motherboard of the device. While the capacity and speed do not match a standard SSD, their performance is adequate for basic computing needs and simple applications.
- External Storage
  - HDDs & SSDs: performance and comparison are identical to the internal versions.
  - Optical discs and optical drives (CD, DVD, blu-rays): cost of an optical disc is low compared to an HDD or SSD and, while their read / write speeds may be slower, they are sufficient for data archiving and playback. However, the discs are prone to scratches, especially if they are not stored correctly, and they require an optical drive to read and write to them.
  - Memory cards (SD, microSD, CF): compact storage devices often used in cameras, smartphones and other portable devices. They are ideal for expanding storage in mobile devices and for storing photos and videos in cameras, using NAND flash memory. Highly durable; read / write times are generally slower than SSDs, but outperform those of optical discs.
  - NAS: dedicated file storage connected to a network that allows multiple users to access data. It is often used in homes or businesses for centralizing data storage, file storage and data backup. Usually made up of multiple HDDs and SSDs configured in RAIDs. It is normally connected to the network via Ethernet, and runs a lightweight operating system designed for file storage, and the management and sharing of files.

Comparison between HDDs and SSDs:

Feature	**HDD (hard disk drive)**	**SSD (solid state drive)**
Storage technology	Magnetic storage with spinning disks and read / write heads	Flash memory with no moving parts
Speed	Slower read / write speeds (generally 50–150 MB/s)	Faster read / write speeds (generally 200–500 MB/s)
Durability	More prone to physical damage due to moving parts	More durable; resistant to physical shock
Noise	Produces noise due to moving parts	Silent operation
Power consumption	Higher power usage due to mechanical parts	Lower power consumption
Cost	Generally cheaper per GB	More expensive per GB
Capacity	Available in larger capacities (up to several TB)	Typically available in smaller capacities (up to several TB, but at a higher cost)
Weight	Heavier due to mechanical components	Lighter
Heat generation	Generates more heat due to moving parts	Generates less heat

A1.1.8 Compression

- Lossy: reduces files by permanently eliminating certain information. This information is redundant or less critical data, resulting in a compressed version that is not identical to the original but is, ideally, indistinguishable from the original to human senses.
  - Transform encoding:



    1. Transform coding takes an image of N × N size and sections it into smaller sub-images of size n × n.
    2. Then the forward transform is carried out on each of the sub-images. The forward transform can use different algorithms, depending on the type of file compression, but for JPEGs DCT (discrete cosine transform) is used. This takes the image data from the spatial domain (pixel values) to the frequency domain. The output breaks the sub-image down into low- and high-frequency coefficients.
       * Low-frequency data: correspond to slow changes in pixel values, such as broad areas.
       * High-frequency data: correspond to rapid changes in pixel values, representing fine details, edges and textures.
    3. These frequency coetficients are then passed to the quantizer. This step significantly reduces file size by simplifying the frequency coefficients obtained from the DCT. The purpose of quantization is to reduce the precision of high-frequency components (the fine details) rather than low-frequency components. This is because the human eye is less sensitive to high-frequency data loss compared to low-frequency detail. The extent of the quantization decides the compression level and the quality of the final image.
    4. The final step of transform coding is the symbol encoder. This is where the quantized coefficients are further compressed using entropy coding techniques. This runs through three further algorithms to reduce the file size by efficiently representing the frequency of occurrence of each symbol. The algorithms used at this stage are
       (a) Zigzag scan
       (b) RLE
       (c) Huffman coding

- Lossless: data is compressed to a smaller size, but can be restored back to the original without any loss of information. This technique works by identifying and eliminating statistical redundancy within the data, and this process can be reversed when needed.
  - Run-length encoding: used to reduce the size of files containing many consecutive repeated characters. The original string of info is represented by the recurrence of the letter + the letter itself. RLE is straightforward to implement and it is very effective for data with lots of repetitions, such as simple graphics and certain types of text file

A1.1.9 Type of services in cloud implementation

- SaaS: delivers software applications over the internet. SaaS provides a cost-effective and convenient solution for businesses and individuals, offering a wide range of applications from productivity tools to customer-relationship management systems.

- PaaS: provides a cloud-based platform that allows developers to build, test and deploy applications without managing the underlying infrastructure. PaaS includes tools & services to facilitate application development, such as databases, middleware (software that connects different applications) and dev frameworks. PaaS accelerates software development by allowing developers to focus on coding rather than infrastructure management.
- IaaS: provides virtualized computing resources over the internet, such as virtual machines, storage and networks. IaaS gives users full control over their virtual machines and networks. This reduces the need for upfront investment in hardware and allows businesses to rent solutions at a lower initial cost using a subscription model. IaaS is also scalable, making it easy to adjust resources as the user base grows.

A1.2 Data representation and computer logic

都是老生常谈的东西，看看就好。金波尔万岁！！！

A1.2.1 Principal methods of representing data

Modern computing ultimately settled on binary (base-2) as the primary number system. The base-2 system represents two possible states: 1 or 0. The binary system is particularly well-suited to represent the state of electrical switches within a computer system: on (1) and off (0). This simplicity reduces hardware complexity and enhances reliability.

The increased reliability of binary systems stems from their use of only two states. Small variations in signal strength do not affect data integrity as much as in systems using larger bases, making binary more robust in noisy (electrical disturbances) environments.

Conversion	Method
Binary → Decimal	Multiply each binary digit by its corresponding power of 2, then add the results. $1011_2 = 1(2^3) + 0(2^2) + 1(2^1) + 1(2^0)$ $= 8 + 0 + 2 + 1 = 11_{10}$
Decimal → Binary	Repeatedly divide the decimal number by 2, record the remainders, and read them from bottom to top. I'd personally recommend the subtraction method. (Subtract the decimal number by powers of 2 and record the results.) If you use this method, remember to start with the most significant bit (MSB) the left most bit, not the least significant bit (LSB) (which is the right most bit.)

Base-2 vs Base-10:

Kibibyte **KiB**	**Mebibyte** **MiB**	**Gibibyte** **GiB**	**Tebibyte** **TiB**	**Pebibyte** **PiB**	**Exbibyte** **EiB**	**Zebibyte** **ZiB**
1 KiB = 1024 bytes	1 MiB = 1024 KiB	1 GiB = 1024 MiB	1 TiB = 1024 GiB	1 PiB = 1024 TiB	1EiB = 1024 PiB	1 ZiB = 1024 EiB
Kilobyte **KB**	**Megabyte** **MB**	**Gigabyte** **GB**	**Terabyte** **TB**	**Petabyte** **PB**	**Exabyte** **EB**	**Zettabyte** **ZB**
1 KB = 1000 bytes	1 MB = 1000 KB	1 GB = 1000 MB	1 TB = 1000 GB	1 PB = 1000 TB	1 EB = 1000 PB	1 ZB = 1000 EB

1 byte = 8 bits, 1 nibble = 4 bits. Note here that KB and Kb are different units. ($KB = 8 \times Kb$)

Hexadecimal: base-16 number system, where 10 - 15 are represented by A to F.
Reasons for using hexadecimal:

- Ease of conversion between binary and hexadecimal
- Compact size of display (used in debugging tools, memory dumps and assembly language programming)

Conversion of hexadecimal ⇔ binary: split the binary in nibbles and turn into hexadecimal for assembling the result. The reverse also stands; i.e. convert each hexadecimal character to binary nibbles, and assemble them in the correct order. (For decimal conversion, I recommend conversion to binary first and then convert to hexadecimal)

A1.2.2 How binary is used to store data

- Characters and strings
  - ASCII encoding: designed to provide a common standard for the interchange of text data. Initially started out as 7-bit encoding system (later 8 bits for more character support). Taking the 8-bit system as an example:
    - 010..... represents uppercase letters, while 011..... represents lowercase letters.
    - The last 5 digits, when converted from binary to decimal, corresponds to the alphabet letters (e.g. 1 → a, 2 → b)
  - Unicode encoding: provides a unique number for every character, regardless of platform, program or language. Few different versions of Unicode:

	UTF-8	**UTF-16**	**UTF-32**
Variable length encoding	1–4 bytes per character	2 or 4 bytes per character	4 bytes per character
Note	Compatibility: backward compatible with ASCII	Surrogate pairs: for characters outside the **Basic Multilingual Plane** (the maost commonly used characters and symbals for almost all modern languages), two 16-bit code units are used	Simplicity: easier to process because each character is exactly 4 bytes
Usage	Most commonly used encoding on the web and in many applications	Often used in Windows and Java environments	Less common due to higher storage requirements

  - Using UTF-8 as an example: (essentially kept ASCII but added some other bytes in a clever way): 11110000 10011111 10011000 10000100
    - 11110000: 1111 means 4 bytes is needed to transmit the character, the 0 coming right after the 4 1s means the end of transmitting the number of bytes. The 3 0s afterwards are the data transmitted.
    - 10.....: 10 means the continuation bytes being sent, the numbers afterwards are transmitted data.
    - 0001 1111 0110 0000 0100 are the final data received.
  - UTF-8 has been adopted by the internet as the main character encoding system. Some problems:

    - Due to the variable length, some characterstake more space compared to single-byte encodings.
    - ... This can lead to larger file sizes in certain contexts.
    - The processing required to handle variable length encoding also requires more complex processing compared to fixed-length systems such as UTF-32.

- Images
  - Bitmap / raster images: reproduce images by using a grid of pixels, with each pixel assigned a specific colour and intensity. Some key concepts:
    - Pixel: short for "picture element", the smallest unit of a digital image / display, representing a single point in the image with a specific color and intensity.
    - Image resolution: the number of pixels contained within a digital image, typically expressed as the dimensions (width × height) in pixels, and sometimes as the pixel density (PPI/ DPI) for print quality.
    - Color / bit depth: the amount of bits used to describe the color
    - Formula for calculating image size:

      $$\text{image size (bits)} = \text{width} \times \text{height} \times \text{bit depth}$$

      However, in reality, this calculation is not completely accurate, as the image would require more data to store metadata and other header information.
    - Metadata: information that describes other data, providing context and details about the data's content, structure and attributes.
  - Ways to improve the image quality:
    - Increase image resolution. A higher resolution allows for greater detail and clarity. The trade-off for a higher resolution image is larger file size, which can impact storage and transfer efficiency. 依旧老生常谈
    - Increase color depth. When we increase the colour depth, it allows for a wider range of colours to be represented, resulting in more vibrant and accurate images. $2^{\text{bit depth}}$ = numbers of color available for representation. Mostly 24 bits for modern photos.
  - When working with graphics, RGB values are often shown in hexadecimal.

- Audio: (analog) a continuous signal that represents sound waves through variations of air pressure. These sound waves can be captured through input devices, such as microphones, which convert the sound waves into a digital signal, which is stored as binary. Process: (金波儿老生常谈)

  1. An analog to digital converter(ADC) samples the amplitude (loudness) of the sound at discrete intervals in a process known as sampling. The rate at which this happens is measured in Hertz (Hz) — the higher the Hertz, the more samples are recorded per second.
     - Amplitude: the magnitude of change in a sound wave, representing the loudness or intensity of the sound.
     - Sampling: the process of converting a continuous analogue signal into a series of discrete digital values by measuring the signal' s amplitude at regular intervals.
  2. This sample 1s then stored and represented as a numerical value in binary. The precision is determined by the bit depth. The larger the bit depth, the more possible values that can be used to describe the sample.
  3. Formula for calculating audio size:

     $$\text{file size (bits)} = \text{sampling rate} \times \text{bit depth} \times \#\text{ of channels}$$

4. Many formats to choose from for storing audio files: (They mainly differ by whether they are compressed or uncompressed)
   - Uncompressed formats store the raw binary data (e.g. WAV, AIFF)
   - Compressed formats use algorithms to reduce the file size for storage or transmission (e.g. MP3, FLAC)

- Video
  - Components include:
    - Frames (visual)
      - In their raw form, frames are stored the same as images, with each pixel having a value that can be represented using a colour model such as RGB.
      - To improve colour efficiency, frames are often converted from the RGB colour model to a different one such as YUV. This helps with compression, as this colour model emphasizes luminance (brightness), which the human eye is more sensitive to than changes in colour detail.
    - Audio tracks
    - Metadata
    - Subtitles and closed captions
  - Compression methods / techniques
    - Spatial compression: particularly effective and commonly used for video that has significant detail variation in each frame. It reduces file size by eliminating redundant information within each frame, such as colour depth or detail levels.
    - Temporal compression: particularly effective and commonly used for video that has consistent motion across frames. It reduces file size by eliminating redundant information between consecutive frames, capturing only the changes or movements from one frame to the next. As a predictive compression technique, it predicts frame content based on the preceding and sometimes following frames, only storing the differences.

Different binary methods for storing integers:

- Unsigned binary (covered in A1.2.1)
- Signed binary (negative + positive):
  - Two's complement: most significant bit represents sign. 0 for positive, 1 for negative. To convert a positive binary number to its negative counterpart in two' s complement, you first invert all the bits and then add 1 to the least significant bit.
  - One' s complement: To obtain the one' s complement of a positive number, you simply invert all the bits (change 0s to 1s and 1s to 0s). One's complement have TWO (positive and negative) zeros.
  - Sign-magnitude: most significant bit serves for sign; the remaining bits represent the magnitude of the number, like how unsigned binary numbers work.
- Binary-coded decimal: method of representing decimal numbers where each digit of the decimal number is encoded separately into its own binary form. BCD assigns a 4-bit binary code to each decimal digit (0-9).
- Gray code (reflected binary code): a binary system where two successive values are only allowed to differ by one bit.

Decimal	3-bit binary	3-bit Gray code
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

For example, from decimal 3 to 4, ordinary binary changes from 011 to 100 (three bits change), while Gray code changes from 010 to 110 (only one bit changes).

- Excess-N (biased representation): a fixed bias (N) is added to the actual value to form an encoded value, and you subtract this bias to decode it. This is used to make all signed integers appear as non-negative binary numbers to allow for easier comparisons and arithmetic operations. 我打算法题的时候有一次这么干过 XD
- Fixed-point representation: a method used to store real numbers (numbers with fractional parts) in binary by fixing the position of the binary point. In a fixed-point system, the binary point is placed at a predetermined position, either between certain bits or at a specific location in the binary sequence. This allows for a straightforward representation of fractional numbers, though with some trade-offs in terms of precision and range.
- Floating-point representation: a method used to represent real numbers that can have a very large range or fractional parts. It does this by storing numbers in a format that includes a sign, an exponent and a mantissa (or significand).

  1. Sign bit (one bit): The sign bit determines whether the number is positive or negative.
  2. Exponent (eight bits): This is used to scale the number by a power of two and is stored using the Excess-127 system in its "biased" form.
  3. Mantissa (23 bits): This represents the significant digits of the number. The mantissa does not store leading ones (in normalized form, $1.xxxxxx \times 2^n$).

  An example of FP representation (32-bit single precision): represent 13.25_{10}.

  1. Convert to binary and normalize:

  $$13.25_{10} = 1101.01_2 = 1.10101_2 \times 2^3.$$

  2. Sign bit: the number is positive, so the sign bit is `0`.
  3. Exponent: add the bias of 127 to the actual exponent, giving $3 + 127 = 130_{10} = 10000010_2$.
  4. Mantissa: omit the implicit leading 1 and store the fractional bits `10101`, padded with zeros to 23 bits: `10101000000000000000000`.

  The stored representation is:

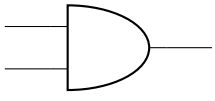
  $$\underbrace{\texttt{0}}_{\text{Sign (1 bit)}} \mid \underbrace{\texttt{10000010}}_{\text{Exponent (8 bits)}} \mid \underbrace{\texttt{10101000000000000000000}}_{\text{Mantissa (23 bits)}}.$$

  To decode it, subtract the exponent bias and restore the implicit leading 1:

  $$(-1)^0 \times (1 + 2^{-1} + 2^{-3} + 2^{-5}) \times 2^{130-127} = 1.65625 \times 8 = 13.25.$$

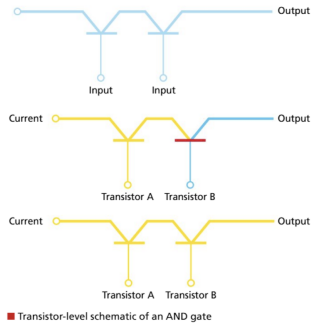
A1.2.3 Purpose and use of logic gates

Logic gates are made up of transistors, which act as electronic switches, allowing or blocking the flow of electrical current. In a transistor, the control wire (or "gate") regulates the current between the two electrodes, known as the "source" and the "drain" . When voltage is applied to the gate, it allows current to flow from the source to the drain, enabling the transistor to switch states and perform logical operations.

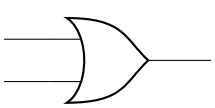


AND
$Y = A \cdot B$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

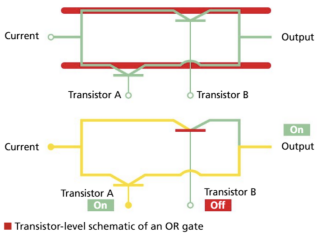


Transistor-level schematic of an AND gate

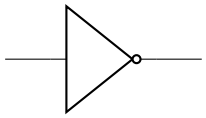


OR
$Y = A + B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

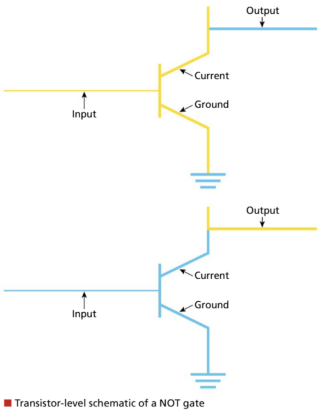


Transistor-level schematic of an OR gate

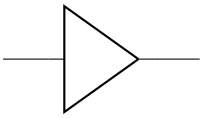


NOT
$Y = \overline{A}$

A	Y
0	1
1	0

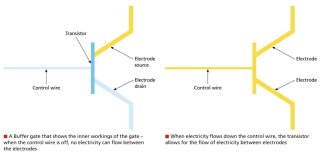


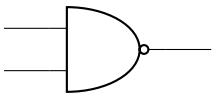
Transistor-level schematic of a NOT gate



BUFFER
$Y = A$

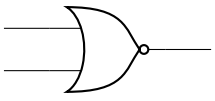
A	Y
0	0
1	1





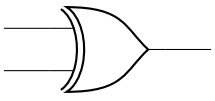
NAND
$Y = \overline{A \cdot B}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



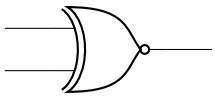
NOR
$Y = \overline{A + B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0



XOR
$Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



XNOR
$Y = \overline{A \oplus B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

A1.2.4 Constructing and analysing truth tables

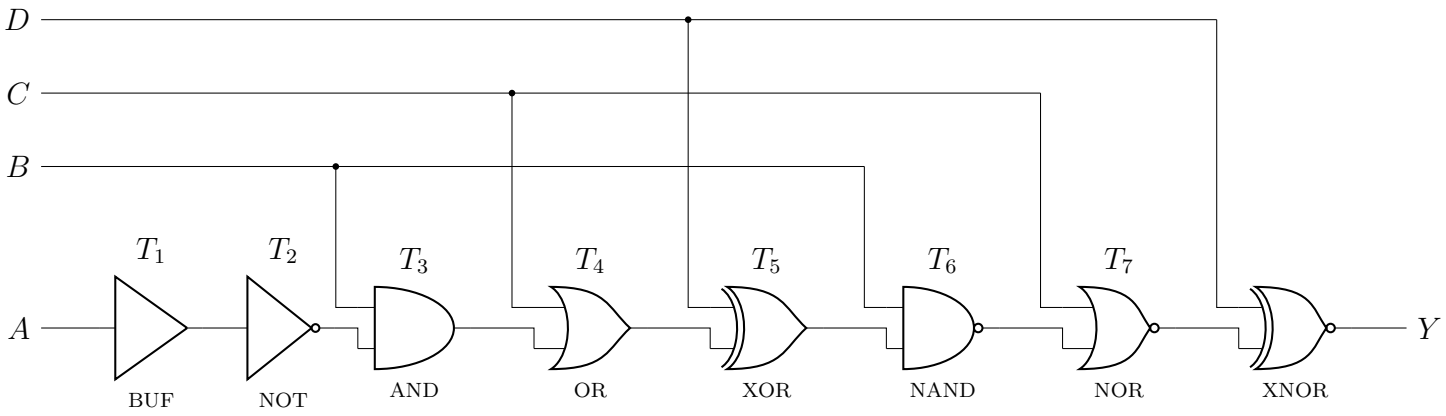
Starting with the intermediate values, work through the logic circuit.
Number of rows in truth table: $2^{\#\text{ of inputs}}$
Worked example: four inputs, all eight gate types.

Each gate output is labelled; every label becomes a column of the truth table.

$$
\begin{aligned}
T_1 &= \mathrm{BUF}(A) = A && \text{(BUFFER)}\\
T_2 &= \overline{T_1} = \overline{A} && \text{(NOT)}\\
T_3 &= T_2 \cdot B = \overline{A}\,B && \text{(AND)}\\
T_4 &= T_3 + C = \overline{A}\,B + C && \text{(OR)}\\
T_5 &= T_4 \oplus D = (\overline{A}\,B + C) \oplus D && \text{(XOR)}\\
T_6 &= \overline{T_5 \cdot B} && \text{(NAND)}\\
T_7 &= \overline{T_6 + C} && \text{(NOR)}\\
Y &= \overline{T_7 \oplus D} && \text{(XNOR)}
\end{aligned}
$$

Standard gate operators $X \uparrow Y = \overline{X \cdot Y}$ (NAND), $X \downarrow Y = \overline{X + Y}$ (NOR) and $X \odot Y = \overline{X \oplus Y}$ (XNOR),

$$Y = \Big(\big(((\overline{A} \cdot B) + C) \oplus D\big) \uparrow B\Big) \downarrow C \;\odot\; D.$$



Inputs				Intermediate values							Output
A	B	C	D	T_1 BUF	T_2 NOT	T_3 AND	T_4 OR	T_5 XOR	T_6 NAND	T_7 NOR	Y XNOR
0	0	0	0	0	1	0	0	0	1	0	1
0	0	0	1	0	1	0	0	1	1	0	0
0	0	1	0	0	1	0	1	1	1	0	1
0	0	1	1	0	1	0	1	0	1	0	0
0	1	0	0	0	1	1	1	1	0	1	0
0	1	0	1	0	1	1	1	0	1	0	0
0	1	1	0	0	1	1	1	1	0	0	1
0	1	1	1	0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0	0	1	0	1
1	0	0	1	1	0	0	0	1	1	0	0
1	0	1	0	1	0	0	1	1	1	0	1
1	0	1	1	1	0	0	1	0	1	0	0
1	1	0	0	1	0	0	0	0	1	0	1
1	1	0	1	1	0	0	0	1	0	1	1
1	1	1	0	1	0	0	1	1	0	0	1
1	1	1	1	1	0	0	1	0	1	0	0

Truth tables to determine outputs from inputs for a problem description: With a problem description, we first need to identify the inputs. Here we have three: alarm switch, baby crying and room temperature, which we can represent as A, B and C and set up our truth table.

Logical expressions: We can represent logic circuits using Boolean algebra. In Boolean algebra, parentheses are often used to indicate which operations should be performed first. If no parentheses are present, follow this sequence: NOT, AND, OR.

Karnaugh maps and algebraic simplification
Two input K-map: the four cells hold the output for the four possible combinations of A and B. For example, the function $F = A + B$ gives

A \ B	0	1
0	0	1
1	1	1

The two 1s in the column $B = 1$ group to give B, and the two 1s in the row $A = 1$ group to give A, so the K-map simplifies to $F = A + B$.

Three input K-map: the eight cells hold the output for the eight combinations of A, B and C, with the column labels in Gray-code order $00, 01, 11, 10$ so that adjacent cells differ by one variable. For example, the function $F = AB + C$ gives

C \ AB	00	01	11	10
0	0	0	1	0
1	1	1	1	1

The four 1s in the row $C = 1$ (that is, all four columns) group to give C, and the two 1s in the column $AB = 11$ (that is, $C = 0$ and $C = 1$) group to give AB, so the K-map simplifies to $F = AB + C$.

Another three input K-map: the function $F = \overline{A}BC + A\overline{B}C + ABC$ gives

C \ AB	00	01	11	10
0	0	0	0	0
1	0	1	1	1

The two 1s in the row $C = 1$ with $B = 1$ (that is, 01 and 11) group to give BC, and the two 1s in the row $C = 1$ with $A = 1$ (that is, 11 and 10) group to give AC, so the K-map simplifies to $F = BC + AC$.

Grouping conventions

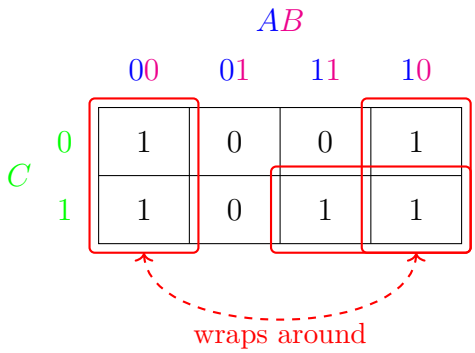
- Groups must contain powers of 2: One, two, four, eight or sixteen 1s can be grouped together.

- Groups must be rectangular or square: Each group should form a rectangle or square shape.
- Groups cannot be diagonal: Adjacent 1s can only be grouped horizontally or vertically, not diagonally.
- Groups must be as large as possible: Always aim to make the largest groups to simplify the expression further.
- Groups can overlap: Some 1s may be included in more than one group if it helps form larger groups.
- Minimize the total number of groups: The goal is to use the smallest number of groups to cover all the 1s.

To determine the expression from the groups, we look at each group of cells and refer to the variables. If a variable' s value stays the same across all the cells in the group, we keep that variable in the simplified expression. Iowever, if the variable' s value changes across the group, we discard it from the expression.

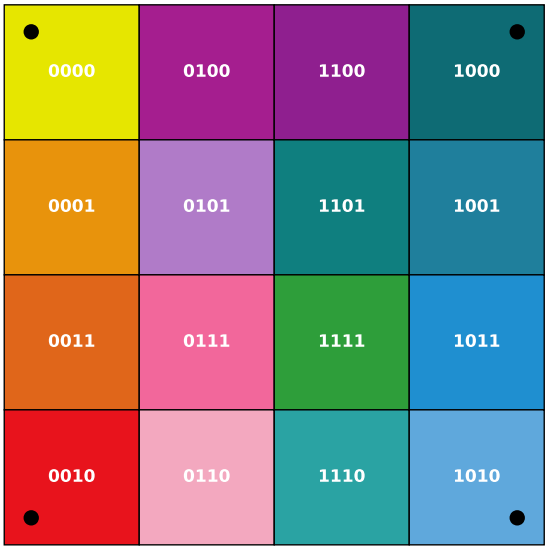
Wrapping around edges in K-maps

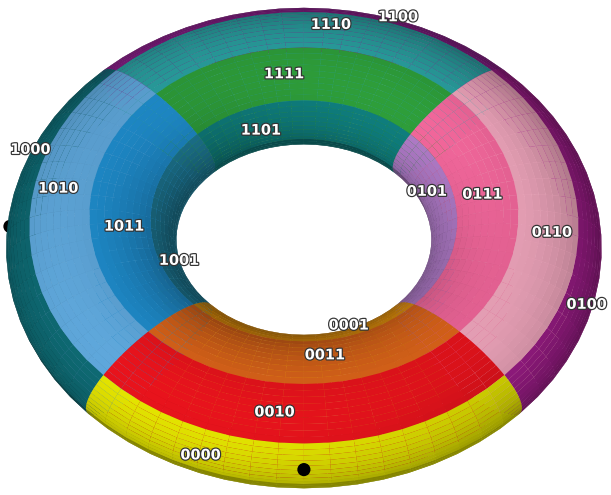
K-maps are considered three-dimensional, and groups can be formed from left to right and top to bottom (although only left to right is possible with three inputs). In the example of $F = \overline{B} + ABC$:



The four 1s in the columns $AB = 00$ and $AB = 10$ wrap around the left and right edges to form one group of four in which $B = 0$ throughout, giving $\overline{B}$; the remaining 1 groups with the adjacent 1 in $AB = 10$ (overlapping the $\overline{B}$ group) to give AC, so the K-map simplifies to $F = \overline{B} + AC$.

Four-variable K-maps wrap in both directions, so the map behaves like a torus: the left and right edges join, and the top and bottom edges join as well. The figure shows a 4×4 map and the same map drawn on a torus; the four wrapping corner cells are marked with dots.





A1.2.5 Constructing logic diagrams

By understanding the principles of Boolean algebra, we can simplify complex logic expressions and translate them into circuit diagrams. This journey from abstract mathematical notation to circuit design is essential for creating efficient and reliable digital systems. When a logic expression is given, follow the conventional operation sequence (parentheses first, then NOT, AND, OR...)

A1.3 Operating systems and control systems

A1.3.1 The role of operating systems

An operating system (OS) is the fundamental software that manages computer hardware and software resources and provides common services for computer programs. It acts as an intermediary between the user and the computer hardware, ensuring efficient and secure operation of the system. Operating systems simplify user interactions with computer hardware by **abstraction**: users and applications do not need to understand the detailed workings of components such as CPUs, memory and I/O devices, because the OS provides a set of high-level services and interfaces that hide these complexities, making the system easier to use and to program.

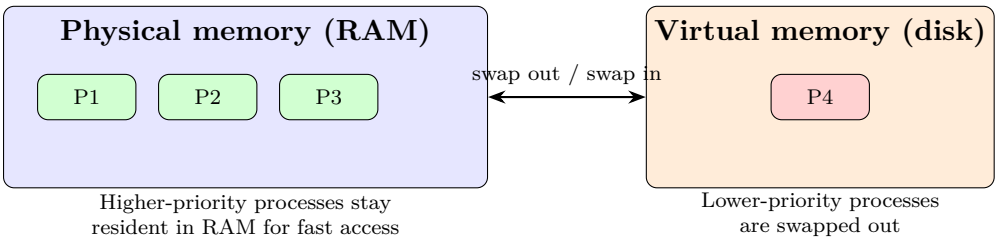
- CPU management: allocating CPU time to various processes and ensuring efficient execution
- Memory management: handling the allocation and de-allocation of memory to applications, and managing virtual memory to extend physical memory capacity
- Storage management: organizing and managing data on storage devices, ensuring reliable data storage and retrieval
- Device management: controlling and co-ordinating hardware devices, providing drivers and interfaces for seamless operation.

A1.3.2 Functions of an operating system

- Memory management: control and coordination of a computer's primary memory; the OS ensures that memory is allocated efficiently to processes and applications, maintains system stability and protects memory areas from unauthorized access. When an app is opened, the OS will load it into RAM. How the memory allocation mechanism works:

  1. Locate the app on the storage device and read the executable file, containing all the app's code and initial data.
  2. The OS then allocates the necessary memory space for the application. This includes space for the code, data and any other required resources. The OS ensures that the application has enough space to execute without interfering with other processes.
  3. Once the application is loaded into RAM, the OS continuously manages memory to ensure efficient operation and system stability.
     (a) While the application is running, the OS can dynamically allocate and de-allocate memory as needed by the application.
     (b) The OS also ensures that each process operates within its own memory space. This prevents processes from interfering with each other, which enhances the security and stability of the system.
     (c) If the applications require more memory than is available in the RAM, the OS can use virtual memory to keep the system running smoothly. (Virtual memory: a memory-management technique that allows a computer to use more memory than what's physically available by temporarily transferring data from RAM to disk storage, enabling the execution of larger

programs and multitasking.) This is not ideal, since reading from a storage device is much slower than accessing RAM, but by switching higher-priority processes into RAM and lower-priority ones into virtual memory, the OS keeps the whole system running at an optimal level.



- File management: involving the storage, retrieval, organization and manipulation of data on storage devices. The OS provides a structured way to manage files and directories, ensuring system stability and security.

  1. The OS employs a hierarchical file system to organize files in a logical and structured manner. Files are organized into directories (or folders), which can contain further directories, creating a tree-like structure. This organization makes it easier for users and applications to locate and manage data.
  2. The OS allows a number of set file operations that users and applications can perform on the files and directories, including creating new files and directories; reading and writing data to files; deleting files; renaming files; and moving or copying files and directories to different locations.
  3. To ensure consistency and avoid conflicts, the OS enforces rules for naming files, ensuring no two files in the same directory have the same name. Files often have a **file extension** — a suffix at the end of the filename that indicates the file type and the program associated with opening or processing that file (e.g. .jpg for images, .exe for applications) — although, depending on the OS settings, these may be hidden from the user.
  4. When managing the storage of files, the OS uses various methods to improve performance: how it allocates files on the physical storage medium, manages free space, and performs maintenance on the saved data.
  5. **Fragmentation** occurs over time as files are created, modified and deleted, causing them to be scattered across different sectors of the disk. **Defragmentation** is the process of reorganizing the data on a hard drive so that files are stored in contiguous blocks, reducing fragmentation and improving access speed and overall system performance.

- Device management: ensuring that hardware devices operate efficiently and interact seamlessly with software applications.

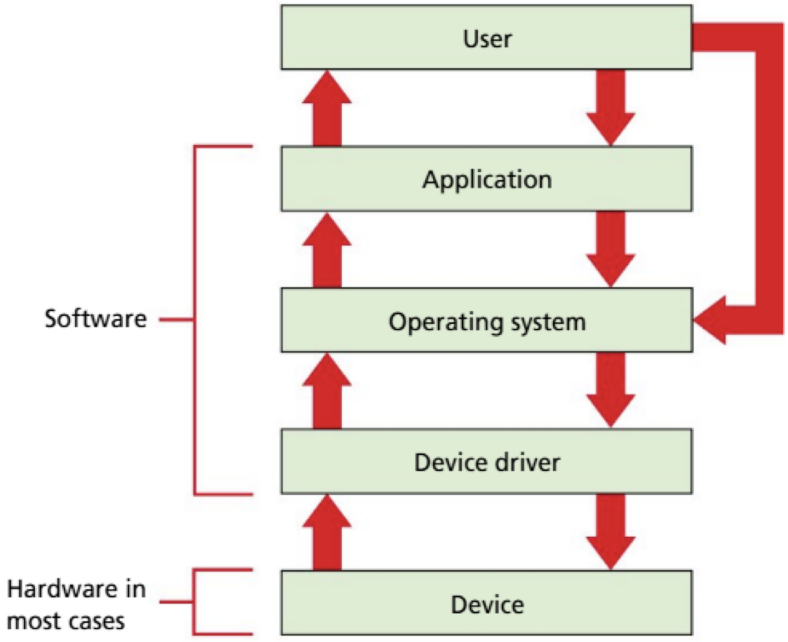
  1. The OS controls and co-ordinates the use of hardware components such as printers, disk drives, display screens, keyboards and network interfaces through specialized software called device drivers. (specialized software programs that allow the operating system to communicate with and control hardware devices, e.g. printers, graphics cards or network adapters, by providing the necessary instructions and protocols.) Each piece of hardware requires a specific driver to function correctly and efficiently, and the OS uses these drivers to provide a **uniform interface**, so that applications can interact with the hardware without needing to understand its specifics.
  2. The OS also handles input/output (I/O) management to co-ordinate data transfer between the computer and peripheral devices. It employs techniques such as buffering, caching and spooling to optimize performance and reliability.

(a) Buffering: the process of temporarily storing data in a memory area (buffer) while it is being transferred between two devices or processes, helping to manage differences in data-flow rates (accommodating speed differences between the CPU and peripheral devices) and ensuring smooth, uninterrupted operation.

(b) Caching: the process of temporarily storing frequently accessed data in a high-speed storage area (cache) to reduce access time and improve system performance by enabling quicker retrieval of the data.

(c) Spooling: the process of queuing data or tasks in a buffer, typically for input / output devices such as printers, which cannot handle interleaved data streams, so that the data is processed sequentially and at the devices' own pace, allowing the system to continue working on other tasks in the meantime.

3. Modern operating systems have introduced user-friendly ways of connecting devices, such as Plug and Play (PnP) technology. With PnP, the OS can automatically identify a device that has been attached and install the necessary drivers and configure settings without the need for user interaction. The OS is also capable of detecting errors and taking action to recover from them, by resetting the device, reinitializing drivers or notifying the user of the issue. Additionally, the OS provides security and access control to ensure that only authorized users and processes can access certain devices, maintaining system integrity and security.



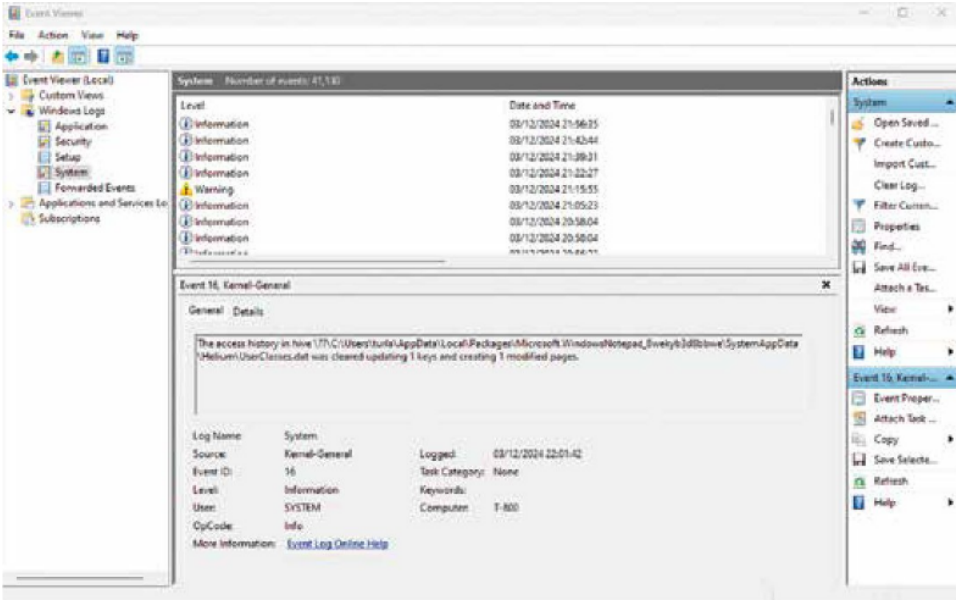
- Scheduling: The OS schedules the process of managing the execution of multiple processes by determining which process runs at any given time. The scheduler is responsible for allocating CPU time to processes, ensuring efficient and fair use of system resources. This function is crucial for multitasking environments, where multiple applications and background processes run concurrently. By implementing various scheduling algorithms, the OS aims to optimize performance, reduce wait times and maintain system stability.

- Security: OS security is designed to protect the integrity, confidentiality and availability of information and resources within the computer system. It has mechanisms to safeguard against threats, prevent unauthorized access and ensure that users and applications can operate securely.

  – User authentication: User authentication can be used in multiple ways. Initially, the OS may require the user to authenticate themselves to log in to the system. It would require the user to provide credentials such as a username and password, biometric data or security tokens. (physical

or digital devices that generate or store authentication credentials, such as one-time passwords or cryptographic keys, used to verify a user's identity and secure access to systems, networks or online services.) Once logged on, based on the user settings, the OS can then grant or prevent access to certain files, folders or applications on the system. This allows systems to have multiple users with different access credentials, such as an administrator and a standard user.

- Encryption: The OS can provide tools and frameworks for encrypting files, communication channels and devices. The encryption ensures that the data, even if it is intercepted or accessed by unauthorized individuals, cannot be read without the decryption key.
- Auditing and monitoring: All system activities, such as users logging in, file access, system errors and administrator actions, are tracked by the OS in a log. These logs are used for auditing and monitoring purposes and can help administrators detect suspicious activities and potential security breaches. They can also be used to help identify issues and areas for improving the system performance.



Windows Event Viewer: the OS records system events (logons, errors, service activity) so that they can be reviewed for auditing and monitoring.

- Malware protection: The OS includes mechanisms to detect and prevent malware infections and intruders. These include antivirus software, firewalls and intrusion detection systems (IDS). These tools scan for malicious software, monitor network traffic and block unauthorized access attempts, protecting the system from viruses, worms, trojans and other malicious threats. (**Malware**: a general term for any software designed with malicious intent, e.g. viruses, worms, trojans, spyware and ransomware, which can damage systems, steal data or disrupt operations. A *virus* attaches itself to legitimate files or programs and spreads to other files or systems, often causing damage or disruption; a *worm* self-replicates and spreads across networks without needing to attach to other programs, exploiting vulnerabilities to infect multiple systems; a *trojan* appears legitimate but carries hidden malicious code, which can create backdoors, steal data or cause harm once the user executes it.)

- Accounting: OS accounting functions are essential for monitoring and managing the usage of system resources. They track resource consumption by users and processes, providing valuable data for system administrators to analyse performance, allocate costs and optimize resource utilization.
  - Resource usage tracking: The OS continuously monitors the consumption of various resources by users and processes; this includes tracking:
    * CPU usage: the amount of CPU time consumed by each process or user.
    * Memory usage: the amount of RAM allocated and used by each process.

    - Disk usage: the amount of storage space occupied by the files and directories owned by each user.
    - Network usage: the volume of data sent and received over the network by each process or user.
  - Process accounting: involves maintaining detailed records of each process that runs on the system; this includes information such as process ID, user ID, execution time, start and end times, resource consumption.
  - User accounting: tracks the resource usage by individual users or user groups; this information is crucial for cost allocation (in multi-user environments such as universities or enterprises, usage data can be used to allocate costs to different departments or projects according to their consumption) and quota management (enforcing resource usage limits so that no single user monopolizes system resources, e.g. disk quotas and memory limits).
  - Performance monitoring: The OS accounting functions are integral to performance monitoring; by analysing resource usage data, system administrators can identify bottlenecks (areas where resources are being overutilized, causing performance degradation), underutilizations (resources that are underused, indicating potential areas for optimization) and trends (patterns in resource usage over time, which can inform capacity planning and system upgrades).
  - Auditing and reporting: The OS generates detailed reports based on the collected accounting data; these reports can be used for auditing (ensuring compliance with organizational policies and regulatory requirements by reviewing resource usage and access patterns), security analysis (detecting unusual or suspicious activity by analysing resource usage anomalies) and resource management (making informed decisions about resource allocation, system configuration and future investments).
  - Billing and chargeback: In environments where resource usage needs to be billed to individual users or departments, such as cloud-computing services or academic institutions, OS accounting functions enable usage-based billing (charging users based on their actual consumption, such as CPU hours, memory usage and network bandwidth) and chargeback (allocating costs to different departments or projects according to their resource usage, promoting accountability and efficient resource use).

- GUI: By offering visual elements such as windows, icons, menus and pointers, the OS provides a user-friendly environment for interacting with the computer and allows the user to execute commands, manage files and run applications.
  - User interface elements: The user interface elements provided by the OS allow the user to interact with the system intuitively.
    - Windows display the contents of applications, documents or system information.
    - Icons give a graphical representation to applications, files and system functions, providing quick access to frequently used items.
    - Menus offer lists of commands or options to access various functions, making navigation and operation straightforward.
    - Pointers, usually an arrow or cursor, are controlled with an input device such as a mouse, enabling users to select, drag and interact with GUI elements seamlessly.
  - Application management: The GUI facilitates the management of and interaction with multiple applications, enhancing user productivity and experience. Task switching lets users move quickly between open applications, with features like the taskbar or the application switcher (Alt + tab) enabling efficient navigation; window management helps users organize their workspace by arranging, overlapping and tiling windows, and snapping windows to edges or corners creates a split-screen effect for multitasking; the desktop environment, where users can place icons, shortcuts and widgets, allows for a personalized and organized workspace.

  - File and system management: The GUI simplifies file and system management tasks through visual tools and interfaces. A GUI-based file management tool such as File Explorer allows users to navigate directories, view file properties and perform operations such as copying, moving, deleting and renaming files and folders, and often includes features such as search, sort and filter. The GUI also provides access to system settings and control panels, enabling users to configure hardware, software and system preferences through intuitive graphical interfaces, and drag-and-drop offers a user-friendly method for transferring files and data between applications and directories.
  - Accessibility: The GUI includes features that enhance accessibility for users with disabilities, ensuring that the system is usable by everyone. Screen readers convert text and GUI elements into speech or Braille, helping visually impaired users to navigate the system; high-contrast themes and screen magnifiers improve readability for users with low vision; keyboard shortcuts allow users to perform actions quickly without relying on a mouse or touchpad, benefiting users with limited mobility.
  - Visual feedback: The GUI provides immediate visual feedback to user actions, enhancing the overall user experience. Progress indicators, including progress bars and loading animations, inform users about the status of ongoing operations such as file transfers or software installations; notifications, in the form of pop-up messages and alerts, keep users informed about important events, updates or errors; tooltips — small informative text boxes that appear when users hover over icons or interface elements — provide additional information or guidance.
  - Customization and personalization: The GUI allows users to personalize their user environment to suit their preferences, making the system more enjoyable and efficient to use. Users can change the appearance of the GUI by selecting different themes, wallpapers and window styles, and widgets and gadgets — small applications that provide quick access to information and tools such as weather updates, calendars and system monitors — enhance the functionality and aesthetics of the desktop.

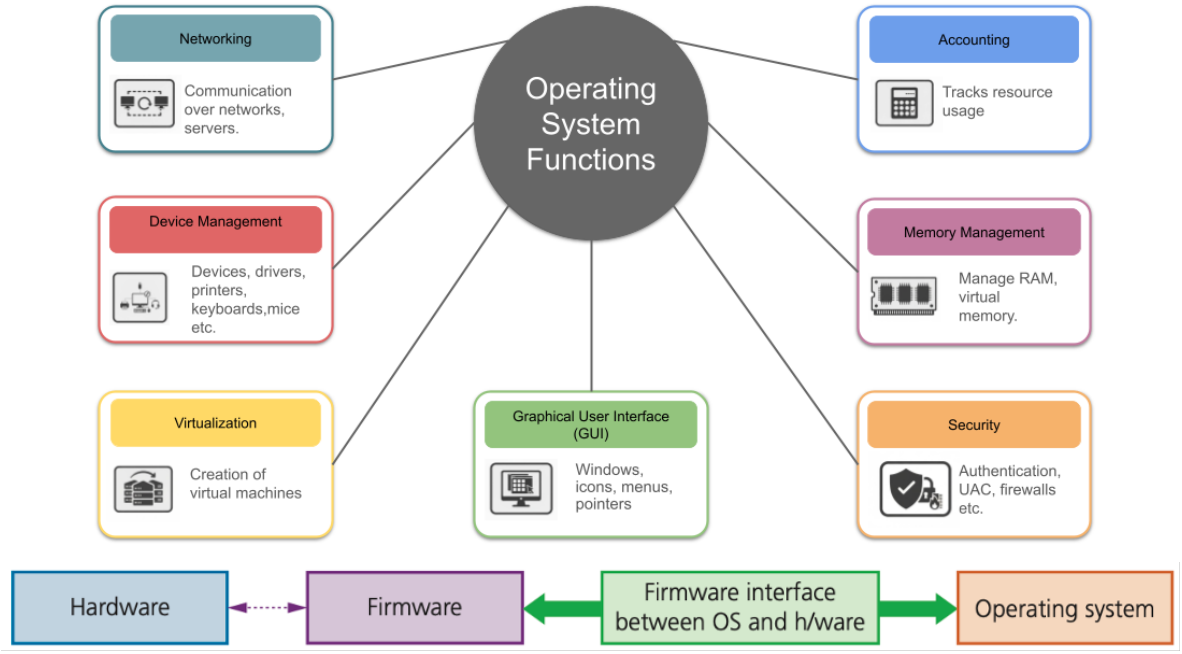
- Virtualization: key feature of modern operating systems that allows multiple virtual machines (VMs) to run on a single physical machine. Each VM operates independently, with its own OS and applications. The operating system manages this through a hypervisor, which allocates CPU time, memory and storage to each VM, ensuring efficient use of resources and allowing each VM to function as if it were on a separate physical machine. Virtualization enhances security and stability by isolating VMs from each other, preventing issues in one VM from affecting the others; it also allows for snapshots and backups, so administrators can save the current state of a VM and revert to it if necessary, and live migration moves VMs from one physical host to another without interrupting services, which aids load balancing and hardware maintenance. Because VMs are easy to back up and restore, virtualization supports disaster recovery and cloud services, and enables the dynamic scaling of IT infrastructure in cloud environments.

- Networking: Operating systems manage and facilitate network communication. One of the primary functions of an OS in networking is to establish and maintain network connections. The OS manages network interfaces and protocols, enabling devices to connect to local area networks (LANs), wide area networks (WANs) and the internet. It configures network settings, assigns IP addresses through DHCP (Dynamic Host Configuration Protocol) and handles the underlying hardware, such as network interface cards (NICs), ensuring that devices can communicate effectively.

  - Protocols and data transmission: the OS implements networking protocols such as TCP/IP (Transmission Control Protocol/Internet Protocol), which govern how data is packetized, addressed, transmitted, routed and received. It ensures data integrity and efficient transmission by handling error checking, flow control and congestion avoidance, and supports higher-level protocols and services such as HTTP for web browsing, FTP for file transfers and SMTP for email communication.
  - Security and access control: the OS employs firewalls, encryption and authentication mechanisms to protect data as it travels across networks. Firewalls monitor and control incoming and outgoing

network traffic based on predetermined security rules, helping to prevent unauthorized access and attacks, and virtual private networks (VPNs) encrypt data and create secure connections over public networks, ensuring privacy and security for remote users.

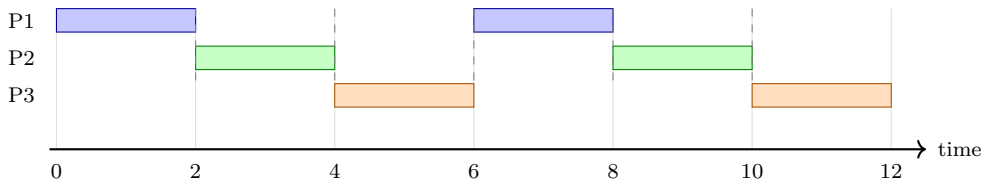
An overview



A1.3.3 Approaches for scheduling

An operating system needs scheduling methods to efficiently manage the execution of multiple processes, ensuring optimal use of the CPU and other resources. Without effective scheduling, processes could experience significant delays or monopolize resources (domination of system resources by a single process, reducing available resources for other processes, thereby slowing system down / inefficiency). The scheduler decides the order in which processes are granted CPU access and for how long, balancing the need for quick response to interactive applications against the efficient processing of long background tasks, so that the whole system stays responsive.

- **First Come First Served (FCFS)**: processes are executed in the order they arrive in the ready queue. A process is added to the end of the queue on arrival; the CPU scheduler takes the process at the front and runs it until it completes or moves to an I/O wait state.
  - Advantages: simple and easy to implement; fair, since requests are served in the order they arrived.
  - Disadvantages: the *convoy effect* — a short process can be stuck behind a long-running one, so waiting times increase and throughput falls; it is non-pre-emptive, so once a process has the CPU it cannot be interrupted, wasting CPU time while it waits for I/O.
- **Round Robin (RR)**: a pre-emptive, time-sharing algorithm. Every process is given a fixed time slice called a *time quantum*; when the quantum expires the CPU pre-empts the process, moves it to the back of the ready queue and starts the next one. The cycle repeats until all processes finish.
  - Advantages: fair allocation of CPU time between processes; high responsiveness and improved system interactivity; prevents any single process from monopolizing the CPU.
  - Disadvantages: the quantum length is critical — too short and the system spends most of its time context switching, too long and it degenerates into FCFS; processes that complete within a single quantum still cost a full switch.



Round robin, time quantum = 2: each quantum expires, the process is pre-empted and moved to the back of the ready queue.

- **Priority scheduling**: each process is assigned a priority level and the CPU is allocated to the highest-priority ready process (equal priorities are broken by FCFS). Priority may be *static* (fixed when the process is created) or *dynamic* (changed over time, e.g. through ageing).
  - Advantages: important and critical tasks are executed as soon as possible, improving responsiveness; flexible, because priorities can be adjusted dynamically as system needs change.
  - Disadvantages: low-priority processes risk *starvation* if high-priority processes continually take the CPU — *ageing* counteracts this by gradually increasing the priority of waiting processes; *priority inversion* can arise when a lower-priority process holds a resource that a higher-priority process needs; more complex to implement and manage than FCFS.
- **Multilevel queue scheduling**: the ready queue is partitioned into several separate queues, each with its own priority level and its own scheduling algorithm. Processes are permanently assigned to a queue by characteristics such as process type, priority or memory requirement; for example interactive processes sit in a high-priority queue scheduled with round robin, while batch processes sit in a lower-priority queue scheduled with FCFS. The scheduler always selects from the highest-priority queue, dropping to lower ones only when the higher queues are empty.
  - Advantages: flexible handling of different types of processes; prioritizes critical and interactive processes, improving responsiveness; each queue can use the algorithm that best suits it.
  - Disadvantages: complex implementation and management; starvation of lower-priority queues if the higher ones are frequently occupied; permanent queue assignment may be sub-optimal if a process's behaviour changes over time.
- **Context switching**: when the CPU switches from one task to another, the state of the running process is saved and restored. It is what makes multitasking possible, but excessive switching reduces overall efficiency because the CPU spends its time saving and restoring tasks instead of doing useful work.

Comparison chart

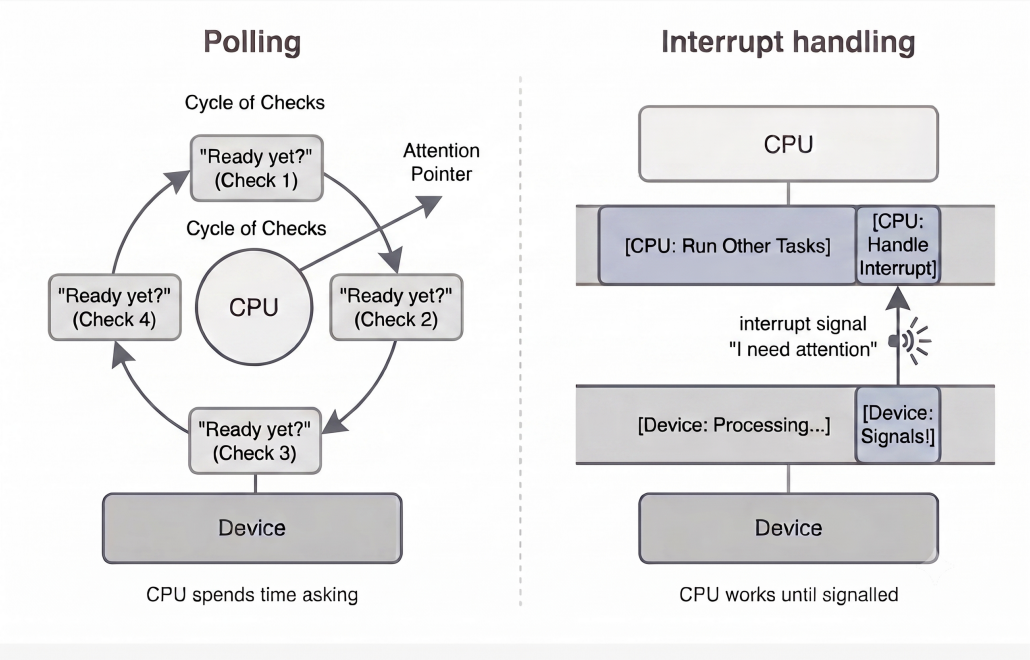
Approach	**How processor time is allocated**	**Main strength**	**Main limitation**
First-come first-served	In arrival order, each process runs to completion	Simple and predictable	Convoy effect: a long job delays all jobs behind it
Round robin	Fixed time slice each, then rotate to the back of the queue	Responsive; no process is starved	Frequent context switching adds overhead
Multilevel queue	Processes split into queues by type or priority, each queue with its own rule	Different work gets suitable treatment	Lower queues can starve; more complex to manage
Priority scheduling	Highest priority ready process runs first	Urgent and real-time work served first	Low priority processes can starve without ageing

A1.3.4 Interrupt handling and polling

Interrupt handling and polling are the two fundamental techniques for managing communication between the CPU and peripheral devices. Each has its own advantages and drawbacks, and which one is better depends entirely on the context of the system.

- **Interrupt handling**: a peripheral device signals the CPU to request service. When an event occurs — a keypress, data arriving on a network interface, a disk completing a read — the device sends an *interrupt signal*. The CPU pauses its current operations, saves its state and executes an *interrupt service routine (ISR)* that addresses the event, before returning control to the main program. The CPU can therefore remain idle or perform other tasks until an event actually occurs.
  - Advantages: highly efficient when events happen sporadically or unpredictably, because the CPU only deals with events when they occur and spends no cycles checking for them; preferred in real-time and interactive systems where an immediate response to events is crucial.
  - Disadvantages: frequent interrupts introduce processing overhead through the context switching involved — each ISR requires the CPU to save its current state and later restore it, which is time-consuming and resource-intensive if interrupts are too frequent; a high volume of interrupts also increases power consumption, which matters on battery-powered devices; if interrupts are too frequent, response times become unpredictable.
- **Polling**: the CPU periodically checks each peripheral device to see whether it requires attention, at predetermined intervals.
  - Advantages: straightforward and simple to implement; provides controlled latency with predictable response times, since checks occur at regular intervals; suitable for real-time applications whose events occur at regular, predictable intervals; adequate for simple embedded systems that are mostly idle.
  - Disadvantages: significant CPU processing overhead, because the CPU repeatedly checks devices instead of doing useful work; resource-intensive and less power-efficient, since the CPU stays active continuously rather than idling; if the polling interval is too long, the detection of critical events is delayed.

Aspect	**Interrupt handling**	**Polling**
Event frequency	Efficient for infrequent, unpredictable events; the CPU responds only when an event occurs	Better for regular, predictable events; the CPU checks at set intervals regardless of whether anything happened
CPU overhead	Low for infrequent events, but rises with high-frequency interrupts due to context switching	Higher: constant device checking consumes cycles even when no event occurs
Power source	More power-efficient, especially on battery-powered devices, because the CPU idles until an event	Less power-efficient, because the CPU stays active continuously checking
Latency	Can respond quickly, but if interrupts are too frequent the response time becomes variable	Controlled, predictable latency, as checks occur at regular intervals
Security	Can react quickly to critical events, reducing the window for malicious activity	Less secure if the polling interval is too long, since critical events are detected late



Choosing between them (it is context, not one being better):

- *Mouse and keyboard*: each keystroke generates an interrupt, so input is processed in real time with instant feedback; polling would force the CPU to check the devices continuously, wasting processing overhead and battery life, and could delay responses. In a simple kiosk or data-entry terminal that is mostly idle, polling at regular intervals is sufficient and simplifies the design.
- *Network communication*: a network interface card (NIC) interrupts the CPU when data packets arrive, so data is received and passed to the application quickly, which is what keeps an audio/video conference at minimal latency; polling would mean constantly checking the NIC, increasing overhead and risking missed packets. Where traffic is minimal and predictable, such as a remote monitoring station sending small packets periodically, polling is more efficient.
- *Disk input/output*: the disk interrupts the CPU when a read or write completes, so the CPU can execute other tasks while the operation finishes; polling would make the CPU waste cycles repeatedly checking for completion during lengthy disk operations. A data logger that writes at fixed intervals may find polling simpler.
- *Embedded and real-time systems*: an automotive airbag sensor must interrupt the CPU immediately on impact, since a delayed response would be catastrophic; a climate-control system that samples temperature at regular intervals is a good candidate for polling, and an industrial robot performing repetitive fixed-interval tasks may prefer polling because it avoids the context-switching overhead and gives more predictable timing.

A1.3.5 The role of the operating system in managing multitasking and resource allocation

The OS plays a critical role in managing multitasking and resource allocation, ensuring that multiple processes can run concurrently and efficiently. *Multitasking* allows a system to perform multiple tasks seemingly simultaneously by quickly switching between them, while *resource allocation* ensures that each task receives the resources it needs (CPU time, memory, I/O devices) to execute properly.

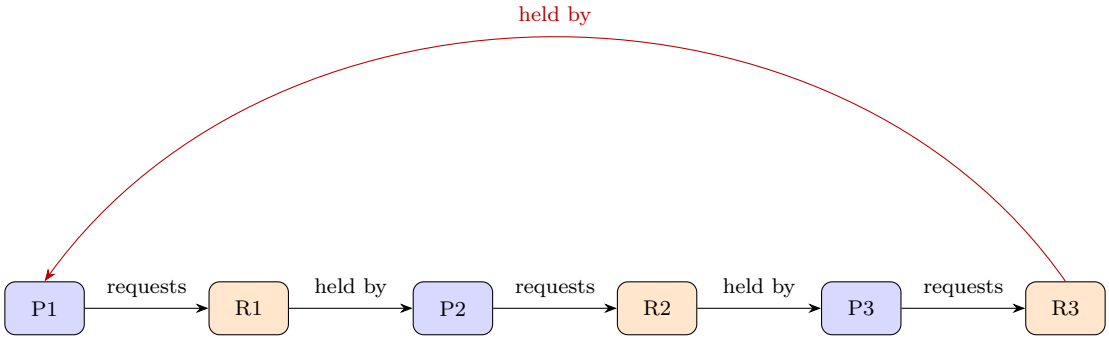
- **Task scheduling**: one of the primary responsibilities of the OS in a multitasking environment. The scheduler decides the order in which processes are executed, aiming to maximize CPU utilization and system responsiveness. It must balance the need for quick response times for interactive processes with the efficient processing of background tasks.

- **Resource contention**: occurs when multiple processes compete for the same resources, such as CPU time, memory or I/O devices. The OS must manage this contention to prevent conflicts and ensure fair and efficient resource usage. The key tool is *mutual exclusion* — preventing multiple processes from accessing a shared resource or *critical section* simultaneously, so that only one process can use the resource at a time. This prevents data corruption and ensures consistency. Techniques for achieving mutual exclusion include *semaphores*, *locks* and *monitors*.
  - If contention is managed badly it leads to *resource starvation*, where a process is constantly denied the resources it needs, or *priority inversion*, where a lower-priority process holds a resource that a higher-priority process requires.
- **Semaphores**: synchronization tools used to control access to shared resources in a concurrent system. A semaphore is an integer variable that can be incremented (*signal*) or decremented (*wait*) atomically.

	Binary semaphore (mutex)	**Counting semaphore**
Possible values	Only 0 or 1, so it acts as a lock guaranteeing mutual exclusion	Any non-negative value
Purpose	Ensures that only one process enters the critical section at a time	Manages multiple instances of a resource
Typical use	Two processes writing to the same log file: the semaphore starts at 1 (available); Process A finds it at 1, sets it to 0 and writes; Process B finds 0 and is blocked until A finishes; A sets it back to 1, then B locks it, writes and restores it to 1	Three database connections available: the semaphore starts at 3; each process that takes a connection decrements it by 1 (value reaches 0 when all three are in use); a fourth process must wait; when a process releases a connection it increments the semaphore by 1, freeing the waiting process

- **Locks**: tools that make sure only one process can use a shared resource at a time. If two programs want to write to the same file, the first must *lock* the file before it can start writing; if the file is already locked by another program, the second has to wait until the lock is released. Types include *binary locks* (also called mutexes), which allow only one process at a time, and *readers-writer locks*, which let multiple processes read a resource but only one process write to it.
- **Monitors**: tools used in programming to manage access to shared resources safely. A monitor acts like a container holding shared variables and the code (methods) that works with them; when a process uses a monitor's method, the monitor is automatically locked so no other process can use it until the first one finishes. Monitors also have *condition variables* that let processes wait for certain events to happen and notify others when those events occur, making it easier to co-ordinate tasks between processes safely.
- **Deadlock**: a problem in multitasking systems where processes get stuck because each one is waiting for a resource that another process holds, creating a cycle with no way to move forward.
  - *Prevention*: design the system so that deadlocks cannot happen.
  - *Avoidance*: use a strategy such as the **Banker's algorithm**, which only allocates resources if it can guarantee that all processes can eventually complete without entering an unsafe state.
  - *Detection*: regularly check for stuck processes, then take steps to fix the problem.
  - *Recovery*: stop one or more processes to break the cycle, or reallocate resources differently.
- **Multitasking challenges**: the OS must also manage context-switching efficiently — saving the state of the currently running process so that another process can be executed. Frequent context switches introduce overheads and reduce overall system performance. The OS must also ensure data consistency

and integrity when multiple processes access shared data, by implementing robust synchronization mechanisms that prevent data corruption and stop processes interfering with each other.



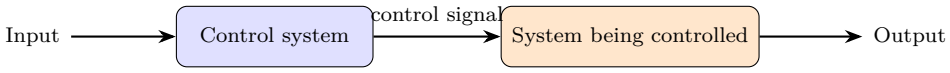
Deadlock: every process holds one resource and waits for the next, so the cycle can never be broken and every process stays blocked forever.

A1.3.6 The use of control system components

Control systems are fundamental in automating and regulating processes across manufacturing, robotics and environmental control. At the core of any control system are components that work together to achieve desired outcomes by managing inputs, processing data and generating outputs, relying on a precise feedback mechanism to ensure the process remains stable and meets its set objectives.

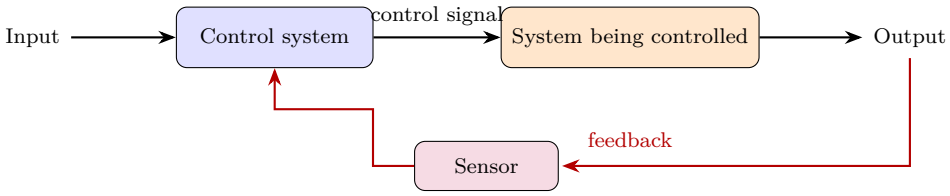
- **Input**: the initial signal or data received by the system, representing the desired condition or target that the system aims to achieve — for example a set temperature in a heating system, or the desired speed in a motor-control application. It is typically generated by a user, another system or an environmental condition, and serves as the reference point for the system's operation.
- **Output**: the result produced by the control system after processing the input; it represents the actual state or action of the system, such as turning on a heater to reach a set temperature or adjusting the speed of a motor. The output is directly influenced by the input and the control process, and is typically the element that can be observed or measured to judge how effective the system is.
- **Feedback mechanism**: the critical component that determines how a control system operates and adjusts itself to maintain desired performance.
  - *Open-loop control*: there is no feedback from the output back to the input or the process. The system operates solely on the initial input, without any correction or adjustment based on the actual output. Simple, and used where the relationship between input and output is straightforward and predictable, such as basic timers or simple heating systems.
  - *Closed-loop control* (feedback control): the system continuously monitors the output and feeds that information back in to adjust the process. If the output deviates from the desired input, the system makes corrections in real time to bring the output back in line with the target. Essential where high accuracy and adaptability are required, such as temperature control in HVAC systems, where heating or cooling must be adjusted from actual temperature readings.

Open-loop control system



There is no feedback path: the output is never measured, so the system cannot detect or correct its own error.

Closed-loop control system



The output is measured and fed back, so any deviation from the target is corrected in real time.

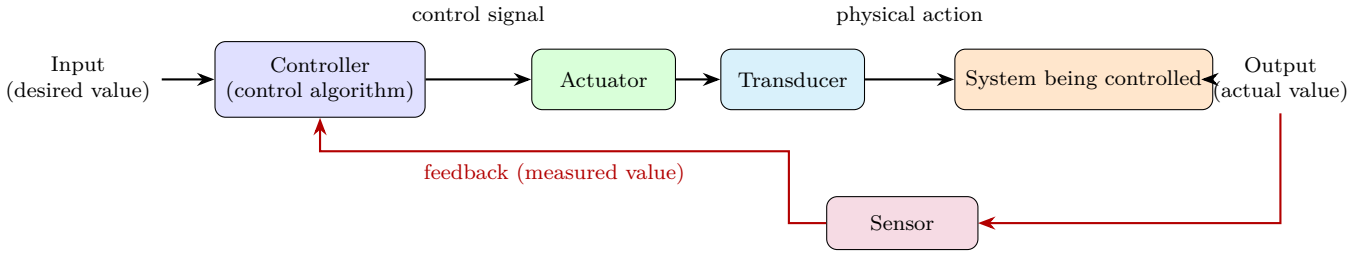
- **Controller**: the central component of a control system; it governs the operation by processing inputs and generating appropriate outputs. It is the "brain" of the system, implementing the control algorithm to make decisions based on the input data and feedback. It compares the input (desired value) with the feedback from the output (actual value) and determines the necessary actions to minimize the difference, or *error*, between them, sending commands to actuators or other components. Controllers range from simple devices such as thermostats to complex microprocessors used in industrial automation.

- **Sensors**: devices that detect and measure physical quantities from the environment or the system itself, such as temperature, pressure, speed or light. The measurements are converted into electrical signals that the controller can interpret. Sensors are the "eyes and ears" of the control system; their accuracy and reliability directly affect performance, because they provide the critical feedback needed to adjust the system's operation — e.g. a temperature sensor in a climate-control system constantly monitors the room temperature so the controller can adjust heating or cooling to maintain the setpoint.

- **Actuators**: the components that carry out the physical actions or adjustments in response to commands from the controller. They convert the controller's electrical signals into mechanical motion or other forms of energy, such as turning a valve, moving a robotic arm or adjusting a motor's speed. Actuators are the "muscles" of the control system, and their performance and precision are critical wherever exact control of movement or processes is required, such as manufacturing equipment and robotics.

- **Transducers**: devices that convert one form of energy into another, typically used to bridge the gap between sensors and actuators and the control system. Often a sensor or actuator cannot directly provide the type of signal the controller can process, or the signal needed to drive the actuator, so a transducer converts them into a compatible form — e.g. a pressure sensor detecting mechanical pressure and converting it into an electrical signal the controller can interpret, or an actuator requiring a specific voltage or current supplied by a transducer. Transducers ensure that all parts of the control system can communicate effectively, enabling accurate and efficient operation.

- **Control algorithm**: the set of rules or mathematical procedures that the controller uses to determine the appropriate output from the input and feedback it receives; it is the logic driving decision-making within the controller. Complexity varies from simple *proportional control*, where the output is adjusted in direct proportion to the error, to advanced methods such as *Proportional-Integral-Derivative (PID)* control, which considers past, present and future errors to make precise adjustments. The choice depends on the system's requirements, such as desired accuracy, speed of response and stability.



The controller compares the desired input with the measured output and computes the error; the actuator applies the correction, while sensors and transducers convert physical quantities into signals the controller can use.

A1.3.7 The uses of control systems

Every one of the following systems is a **closed-loop** system: the sensors continually send updated data back to the controller, and the controller adjusts its actions in real time in response.

System	Input (sensors)	Controller action	Output (actuators)
Home thermostat	Temperature sensor continuously monitoring the room	Compares the current temperature with the desired setpoint and decides whether to activate heating or cooling	Heater, cooler / air conditioner
Automatic elevator	Position sensors, floor requests, door-status sensors	Processes the inputs to decide whether to move up or down, stop at a requested floor, or open and close the doors; adjusts the motor to slow down and halt precisely at the correct floor	Lift motor, door mechanism
Autonomous vehicle	Cameras, LiDAR, radar and GPS, gathering obstacles, road and traffic-signal data	Usually powered by advanced AI; runs control algorithms to make real-time decisions about speed, direction and braking	Steering, acceleration and braking systems
Washing machine	Water-level, load-size and cycle-progress sensors	Fills the drum with water, agitates the clothes and drains the water; stops filling once the required level is reached and adjusts the duration and intensity of the spin cycle to the detected load size	Motor, inlet valve, drain pump
Traffic signals	Sensors detecting vehicles, pedestrians and sometimes traffic conditions	Determines the timing and sequence of the lights (red, amber, green) for each direction; extends a green light for a congested lane or triggers a pedestrian crossing light	Signal lamps
Irrigation control	Soil-moisture, weather and sometimes time-of-day sensors	Decides when and how much water to deliver; activates the system when the soil is too dry and shuts off the supply once the desired moisture level is reached, conserving water and avoiding both under- and over-watering	Pumps and water valves
Home-security system	Door and window sensors, motion detectors, cameras	Assesses potential threats and determines the response, such as sounding an alarm, notifying the homeowner or contacting emergency services; locks doors or activates cameras to record an intrusion	Alarm, cameras, door locks, notification devices

System	Input (sensors)	Controller action	Output (actuators)
Automatic doors	Infrared motion detectors or pressure mats detecting people or objects near the entrance	Decides when to open or close the doors; opens as soon as movement or pressure is detected and signals them to close once the person or object has passed and nothing is detected	Door motor

Exam pointers:

- In every example the *input* is a physical quantity measured by a sensor, the *output* is a physical action performed by an actuator, and the *feedback* is the measured result being returned to the controller.
- The key difference between open-loop and closed-loop shows up clearly here: a closed-loop system (thermostat, elevator, washing machine) can correct itself when the actual value drifts from the target, whereas a purely open-loop system (e.g. a simple timer) cannot.
- Autonomous vehicles and elevators are the best examples to use when explaining why *transducers* matter: sensors such as LiDAR and radar produce signals that the controller cannot use directly, and actuators such as steering and braking need precisely shaped electrical signals, so transducers convert between the physical world and the electronic signals.

A1.4 Translation

A1.4.1 Translation processes of interpreters and compilers

Interpreters and compilers both transform high-level code into machine-readable instructions, yet they do so through different methods, each with unique implications for performance, error handling and development efficiency.

- **Interpreters**: translate high-level programming code into machine code *line-by-line or statement-by-statement*, executing each line as it is translated. Unlike compilers, they do not generate an intermediate machine-code file; instead they directly execute the source code on the fly, reading the code, translating it and running it immediately, and repeating this for each line or block until the whole program has been executed.
  - *Mechanics*: an interpreter can start executing a program without processing the entire codebase upfront, which allows for immediate feedback — particularly useful during development and debugging. The cost is slower overall performance, since the code must be repeatedly translated and executed on every run.
  - *Use cases*: quick testing and debugging are essential; languages such as Python, JavaScript and Ruby are often interpreted, making them popular for web development, scripting and rapid application development. Interpreters are also ideal for educational purposes, enabling beginners to see results in real time, and are favoured where portability matters, since the same source code runs on any platform that has the appropriate interpreter.
- **Compilers**: translate the *entire* high-level source code into machine code in a single, comprehensive process *before* the program is executed. This involves several stages: **lexical analysis** (breaking the source into tokens), **syntax analysis** (checking the tokens form valid code), **semantic analysis** (checking the code actually makes sense), **optimization** and **code generation**. The final output is a standalone executable file, typically in machine code or an intermediate form such as bytecode, which can be run directly on the target machine.

  - *Mechanics*: once compiled, the machine code needs no further translation, so the program executes much faster than interpreted code. However the initial compilation can be time-consuming, especially for large and complex programs, and because the whole code must be compiled before execution, any source errors have to be addressed first, which slows the development cycle.
  - *Use cases*: performance-critical software, where languages including C, C++ and Java (which compiles to bytecode for the JVM) are suited to system software, high-performance computing and real-time systems such as video games. Compiled code is also advantageous where security and resource control matter, since pre-generated, optimized machine code is harder to reverse-engineer and is less dependent on external environments. Finally, compilers are often chosen when software must be deployed across different hardware specifications, since compilation can be tailored to optimize the executable for specific architectures.

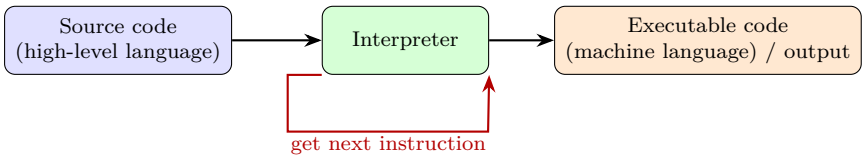
- **Bytecode interpreters**: first translate the high-level source code into an intermediate form known as **bytecode**. This bytecode is not executed by the machine's hardware directly, but by a **virtual machine (VM)** or an interpreter that understands the bytecode's instructions. Bytecode is a compact, platform-independent representation of the program, so it can run on any system with the appropriate VM.

  - *Use cases*: Java is the prime example — code is compiled into bytecode that runs on the Java Virtual Machine (JVM), so the same application runs on any platform with a JVM, making it ideal for enterprise applications, web services and mobile apps. Python uses a similar approach, compiling source to bytecode (the `.pyc` file) which is then executed by the Python interpreter, so bytecode interpreters are also valuable in educational settings, web development and scripting, where flexibility and ease of deployment matter more than raw performance.

- **Just-in-time (JIT) compilation**: a dynamic approach that combines elements of both interpretation and compilation. Initially the source code is compiled into bytecode, which is then interpreted by a virtual machine. As the program runs, the JIT compiler identifies frequently executed sections of the bytecode — the *hot spots* — and compiles them into machine code on the fly. This machine code is then cached, so the next time the same code is executed the system uses the compiled version instead of interpreting it again.

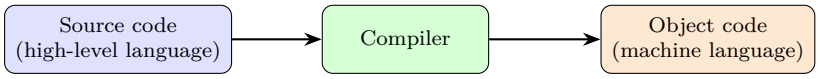
  - *Use cases*: beneficial where performance is critical but the application must also stay portable and dynamically optimized. Both the JVM and the .NET runtime use JIT compilation, and it is especially valuable in long-running applications such as servers, where the overhead of JIT compilation is outweighed by the gains on subsequent runs of the same code paths. Web browsers also use it to optimize frequently executed JavaScript and improve page load times.

Interpreter



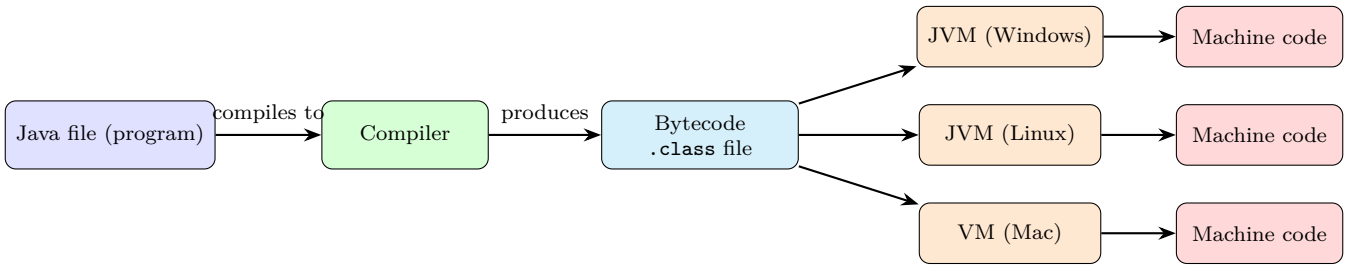
The interpreter reads one statement, translates it, runs it and asks for the next — so the program starts running immediately, without waiting for the whole codebase to be translated.

Compiler



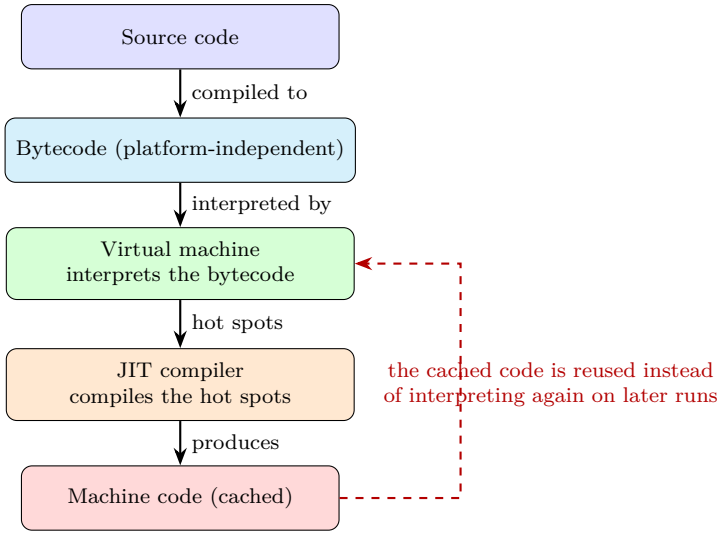
The whole program is translated in one go before anything runs: lexical analysis, syntax analysis, semantic analysis, optimization and code generation produce a standalone executable.

Bytecode interpretation



The bytecode is platform-independent, so the same `.class` file runs on any system with a suitable virtual machine — this is what makes Java applications portable.

Just-in-time (JIT) compilation



Top tip: An interpreter is like someone who translates each sentence of a book for you as you read, giving you immediate understanding, but requiring them to translate each line again every time you revisit it. A compiler is like a translator who first converts the entire book into your native language, letting you read it smoothly without any further translation — but you have to wait until the whole book is translated before you can start reading.

A1.4.2 Evaluating the translation processes

- **Error detection**: compilers offer the most robust detection, because they analyse the entire source code before producing an executable, so all syntax and some semantic errors are caught and must be resolved before the program can run — giving fewer runtime errors and a more stable final product. Interpreters detect errors at run time, as they execute code line by line, which lets developers quickly identify and correct errors during development; the downside is that errors are only discovered when the offending code actually executes, so a runtime error can disrupt execution unexpectedly. Bytecode interpreters provide a middle ground: some errors are caught before execution, but others still appear at run time, reducing the frequency of runtime errors without offering the exhaustive checking of full compilation. JIT combines runtime error detection with dynamic optimization.

- **Translation time**: compilers require a considerable amount of time initially to convert the entire source code, but the payoff is that the compiled program runs significantly faster once translation is complete. Interpreters execute code directly by translating it line by line, so the program runs almost immediately, which is advantageous for quick testing and iterating; however, larger codebases experience slower overall performance. Bytecode interpreters balance the two — compiling to bytecode is quicker than compiling to machine code, and bytecode then executes more efficiently than direct interpretation of source. JIT goes a step further by converting bytecode into machine code dynamically as the program runs: some runtime overhead, but it reduces initial translation time while improving execution speed as the program continues, especially for frequently executed code paths.

- **Portability**: a significant advantage of interpreters and bytecode interpreters. Since interpreters execute source code directly, the same code runs on any platform with the appropriate interpreter, making them ideal for cross-platform applications. Bytecode interpreters extend this by compiling source into platform-independent bytecode that runs on any system with the appropriate VM (for example Java's JVM), which is particularly valuable where applications must operate across diverse platforms without modification. Compilers, however, produce machine code tailored to specific hardware architectures, resulting in highly efficient but far less portable executables — each target platform may require separate compilation. JIT preserves the portability of bytecode while enhancing performance, optimizing for the specific hardware at runtime.

- **Applicability**: compilers are best suited to applications where performance is critical, such as system software, high-performance computing and real-time systems, since the upfront compilation time is justified by the high execution speed of the resulting code. Interpreters are ideal for scenarios where rapid development, testing and iteration are essential — scripting, web development and educational environments — where flexibility and quick feedback matter more than raw performance. Bytecode interpreters are commonly used in enterprise applications, web services and mobile apps where cross-platform compatibility is crucial, offering a flexible balance of portability and efficient execution. JIT compilation is particularly valuable in long-running applications and complex systems where performance needs to improve over time — server environments, dynamic web applications, and platforms such as Java and .NET.

Comparison Chart

	Interpreter	**Compiler**	**JIT compiler**	**Bytecode interpreter**
Error detection	At run time, as each statement is reached	At compile time, before execution	At run time; verification can catch some errors first	At run time; bytecode verification catches some errors first
Translation time	Negligible; starts at once	Slow; whole program translated upfront	Fast start, then compiles hot code during the run	Faster than full compilation; bytecode is an intermediate step
Execution speed	Slower; translation continues throughout	Fast; optimised native code	Near-compiled on hot paths	Faster than direct interpretation, below native
Portability	High; runs anywhere the interpreter exists	Low; native code is platform-specific	Moderate; tuned to the host at run time	High; same bytecode runs on any compatible VM
Applicability	Scripting, prototyping, dynamic languages	Performance-critical and systems software	Balance of portability and speed	Portable apps with moderate performance needs

A1.4.3 Example scenarios

Scenario	**Best translation method**	**Explanation**
Rapid development and testing	Interpreters	Immediate execution allows quick iterations, debugging and real-time feedback; ideal for scripting languages such as Python or JavaScript
Performance-critical applications	Compilers	The whole codebase is optimized into machine code before execution, giving highly efficient, fast-running applications; suitable for system software, gaming and real-time systems in C or C++
Cross-platform development	Bytecode interpreters and JIT compilation	Bytecode gives platform independence (the same code runs on any platform with the right VM), while JIT enhances performance by optimizing frequently executed code at runtime; ideal for enterprise software, mobile apps and web services

A2 Networks

A2.1 Network fundamentals

A2.2 Network architecture

A2.3 Data transmission

A2.4 Network security

A3 Databases

A3.1 Database fundamentals

A3.2 Database design

A3.3 Database programming

A3.4 Alternative databases and data warehouses

A4 Machine learning

A4.1 Machine learning fundamentals

A4.2 Data preprocessing

A4.3 Machine learning approaches

A4.4 Ethical considerations

B Computational Thinking and Problem Solving

B1 Computational thinking

B1.1 Approaches to computational thinking

Scope and reference: Oxford, MacKenty and Stephenson (2025), pp. 317–334; IB Computer Science guide (first assessment 2027), B1.1.1–B1.1.4. These four numbered paragraphs cover B1. Computational thinking is a toolkit for formulating and solving problems; it can be used without writing a program. Application examples below also draw on the classroom material supplied by the student and are expressed as condensed study notes.

B1.1.1 Problem specification

A problem specification defines what needs solving, who needs the solution, the restrictions, and how success will be judged. Agree it with the intended users before designing the solution: otherwise a technically correct program may solve the wrong problem.

1. **Problem statement**: identify the affected users, the current difficulty and its consequences. State what and why, using precise evidence where available. "Improve school administration" is too vague; "staff must compare room-booking records manually, so clashes can be missed" identifies a specific problem.

2. **Constraints and limitations**: state boundaries such as budget, deadline, available hardware, supported software, connectivity and privacy requirements. Distinguish a restriction on development from a limitation of the solution: an offline prototype cannot synchronize bookings across multiple computers.

3. **Objectives and goals**: an objective describes the desired broad outcome; specific goals make it measurable and, where appropriate, time-bound. For example, improve booking reliability; during a two-week pilot, detect every clash in an agreed test set.

4. **Input specifications**: define data items, types, formats, valid ranges, sources and validation rules. For example, a room ID must reference an existing room; a start time uses a 24-hour time format.

5. **Output specifications**: define the result's content, format, presentation and recipient. For example, show a confirmation containing the room, date and time, or an error explaining the clash.

6. **Evaluation criteria (success criteria)**: define testable benchmarks, linked to the original problem and goals, and state how evidence will be collected. Consider effectiveness (solves the problem), accuracy (correct results), efficiency (time/resources), usability (users can complete tasks) and maintainability (changes can be made safely). "Easy to use" is vague; "four of five staff can create a booking unaided within two minutes" is testable.

Goals versus criteria: a goal is a desired result; a criterion supplies the pass/fail measure used to evaluate it. An input or output specification describes required data or behaviour; testing against it provides evidence of success.

Worked real-world problem: avoiding school room-booking clashes

Illustrative specification: the quantities below are proposed requirements, not measurements of an actual school.

- **Problem statement**: teachers book shared rooms through separate messages. Staff cannot reliably see existing reservations, creating the risk of two activities being assigned the same room at the same time. A central booking tool should detect conflicts before accepting a reservation.

- **Constraints and limitations**: build a prototype in four weeks using existing school computers and no paid services. Pilot it on one staff-operated computer with local storage; a shared version would require centralized storage and concurrency control. Store only staff identifiers needed for booking, and restrict access to authorized staff. Assume bookings begin and end on the same date and use one school time zone.
- **Objective and goal**: improve room allocation; by the end of a two-week pilot, prevent all overlapping bookings in an agreed test set and let staff view a room's daily bookings.
- **Inputs**: room ID (string from a maintained list), staff ID (non-empty authorized identifier), booking date (valid date in `YYYY-MM-DD`), start and end times (`HH:MM`, 24-hour format). Require $start < end$. Existing bookings supply room, date, start and end values for comparison.
- **Outputs**: a saved booking with a unique reference and a confirmation showing room/date/times; or a validation/conflict message. A daily schedule lists accepted bookings in start-time order. Rejected requests must not alter stored bookings.

Measurable evaluation criteria and evidence

1. Reject every overlap in tests covering identical times, partial overlaps and one interval containing another; accept adjacent intervals, different rooms and different dates. Compare actual results with predetermined expected results.
2. Reject an unknown room, missing staff ID, impossible date, malformed time and $end \leq start$; display a useful error for each test.
3. After saving and restarting, retrieve every accepted test booking exactly once, with unchanged details; verify rejected requests were not saved.
4. On the pilot computer with 1,000 stored bookings, each of 20 test requests returns a result within one second. Record timings and the test environment.
5. At least four of five staff testers create a valid booking and locate it in the daily schedule unaided within two minutes; observe and record task completion.
6. Change the room list in its configuration without changing the overlap-checking algorithm; rerun the relevant acceptance tests to check maintainability.

How a computer program solves it:

1. Read and validate the requested room, date and times.
2. Retrieve existing bookings for the same room and date.
3. For each booking, test ($newStart < existingEnd$) AND ($newEnd > existingStart$). If true, report a clash and reject the request. Treat intervals as $[start, end)$, so a booking ending at 10:00 does not clash with one starting at 10:00.
4. If no overlap exists, save the booking, assign its reference and output confirmation. Sort the daily schedule by start time when requested.

Why use a program? The input is structured, the conflict rule is precise, and the same comparisons are repeated for every request. A program applies the rule consistently, searches many records quickly, preserves bookings and makes the schedule easy to retrieve. Accurate input and a shared source of truth are still necessary. For simultaneous online requests, checking and saving must be protected as one atomic operation, so two users cannot both reserve the same free slot. Staff still decide priorities and resolve exceptional requests.

B1.1.2 Fundamental concepts of computational thinking

- **Decomposition**: break a complex problem into smaller, manageable subproblems with clear responsibilities and interfaces. Parts can be developed, tested and debugged separately; independent tasks can be assigned to different people. In the room-booking example, separate input validation, conflict checking, storage and schedule display. Dependencies still determine which tasks must happen first.
- **Pattern recognition**: identify similarities, recurring structures or trends in problems, data or solutions, then reuse or adapt a suitable solution. Every booking uses the same interval-overlap test, so one function can handle every room. Repeated demand for a room at lunchtime may also reveal a usage pattern. A pattern suggests a useful generalization; it does not by itself prove a cause.
- **Abstraction**: retain details relevant to a purpose and hide or omit others, producing a simpler representation or interface. Conflict checking needs room ID, date and times, but not wall colour or furniture brand. A `saveBooking` interface hides file-handling details from the conflict checker. Which details matter depends on the task: room capacity matters if capacity checking is added. Abstraction manages complexity; it does not automatically make programs smaller or faster.
- **Algorithmic design**: create a precise, ordered and unambiguous set of steps to solve a problem, with defined inputs, outputs and termination conditions. Use sequence, selection and iteration as required. For booking: validate input, loop through relevant reservations, reject if an overlap is found, otherwise save and confirm. The design can be expressed in plain language, pseudocode or a flowchart before implementation in a programming language.

The concepts work together and may be revisited in any order. Decomposition asks "what smaller tasks?", pattern recognition asks "what is similar?", abstraction asks "what details matter?", and algorithmic design asks "what exact steps?" A person could use the same ideas to organize a paper booking register without programming.

B1.1.3 Applying computational thinking to solve problems

Related reading: InThinking B 1.1.3: Applying computational thinking. The examples below explain both how each concept is applied and why it helps.

Software development — an online shop checkout

- **Decomposition**: separate basket calculation, address validation, stock reservation, payment and order confirmation. Clear interfaces allow a payment provider to be changed without rewriting basket calculations.
- **Pattern recognition**: basket items, saved addresses and order history all require authorized retrieval of a user's records; shared data-access and validation routines reduce duplicated logic.
- **Abstraction**: a payment API accepts an amount and a payment token and returns a result, hiding the provider's internal bank communication and processing.
- **Algorithmic design**: validate basket and address, reserve stock, request payment, record the result, then confirm the order. Handle failure explicitly: release reservations and, if a charge has already succeeded, arrange a refund or other compensating action. An external payment cannot simply be undone by rolling back a local database transaction. This ordering and failure handling reduce inconsistent orders and duplicate charges.

Data analysis — school attendance reporting

- **Decomposition**: import records, validate and clean them, group by student, calculate attendance, identify students requiring review, and generate the report.

- **Pattern recognition**: repeated absences on particular weekdays or in one tutor group suggest questions for staff to investigate. Correlation alone does not establish the reason for absence.
- **Abstraction**: replace thousands of individual marks with each student's ID, expected sessions, present sessions and attendance percentage. Keep the raw records available for checking.
- **Algorithmic design**: standardize IDs/dates/status codes, resolve duplicates and missing or invalid records under an agreed policy, then calculate $100 \times present/expected$ for each student with $expected > 0$. Flag values below an illustrative 85% threshold and display one decimal place. Compare the unrounded value with the threshold; handle zero expected sessions as "not applicable". Standardization should precede duplicate detection when inconsistent formats can hide duplicates. Automated summaries save repetitive work; staff review the context before acting.

Network security — detecting repeated failed logins

- **Decomposition**: separate event collection, parsing, counting, rule matching, alerting and logging. Detection rules can change without replacing the collection system.
- **Pattern recognition**: many failed SSH logins from one source in a short period suggest a brute-force attempt. Signature or anomaly detection can flag suspicious activity, but legitimate failures may produce false positives and unfamiliar attacks may be missed.
- **Abstraction**: show an alert containing source IP, target service, time window and failure count instead of making an analyst inspect every event. A network diagram similarly retains relevant devices, connections and trust boundaries.
- **Algorithmic design**: on each failure, discard events outside the previous 60 seconds, update the source's count, and alert if it exceeds 100. If policy permits, temporarily block the source, log the action and schedule review. Counts require timestamps and a defined window. Consistent automated checks help staff respond quickly, while review accounts for shared IP addresses and false positives.

Database design — a school library

- **Decomposition**: divide the design work into identifying entities and requirements, defining tables/relationships and constraints, then choosing physical storage and indexes. Separate Students, Books and Loans so each represents a clear entity or relationship; Loans references the student and book using keys.
- **Pattern recognition**: seeing student details copied into many loan records suggests redundancy and update anomalies. Examine functional dependencies to decide how to normalize the design; repeated values alone do not prove a table is incorrectly designed.
- **Abstraction**: represent students by ID, name and year group, and loans by borrower, book and dates. Exclude medical details because they do not serve this library's purpose. The schema describes logical data without exposing disk layout.
- **Algorithmic design**: to report overdue loans, combine Loans with Students and Books using their keys, select unreturned loans with a due date before today, and sort by due date. Defined rules produce a repeatable, readable report. SQL describes the required result; the database engine chooses a physical execution plan, which need not follow the written clause order.

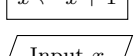
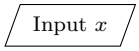
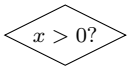
Machine learning — an email spam classifier

Machine learning fits a model to examples so it can make predictions for new inputs, rather than requiring a hand-written rule for every case.

- **Decomposition**: separate labelled-data collection, training/validation/test splitting, preprocessing, feature extraction, training, evaluation and deployment. Fit preprocessing on training data to avoid leaking information from evaluation data.
- **Pattern recognition**: learn statistical associations between spam labels and features such as word frequency, link count and sender characteristics. The aim is generalization to unseen emails; overfitting occurs when a model learns training-specific detail that does not generalize.
- **Abstraction**: represent an email as a numerical feature vector, such as vocabulary frequencies and link count. In a trained linear or neural classifier, learned weights summarize relationships between those features and labels. This description is model-specific: some ML methods retain training examples, and models can memorize information.
- **Algorithmic design**: for a neural classifier, initialize weights, process training batches, calculate loss, use backpropagation to find gradients and update weights; repeat subject to a stopping rule based on validation performance. Evaluate the final model on untouched test data, including false positives and false negatives. At deployment, a chosen threshold (for example, spam probability > 0.85) routes messages to spam or inbox. The threshold trades missed spam against legitimate messages incorrectly filtered; learning and consistent classification reduce manual sorting.

B1.1.4 Tracing flowcharts for programming algorithms

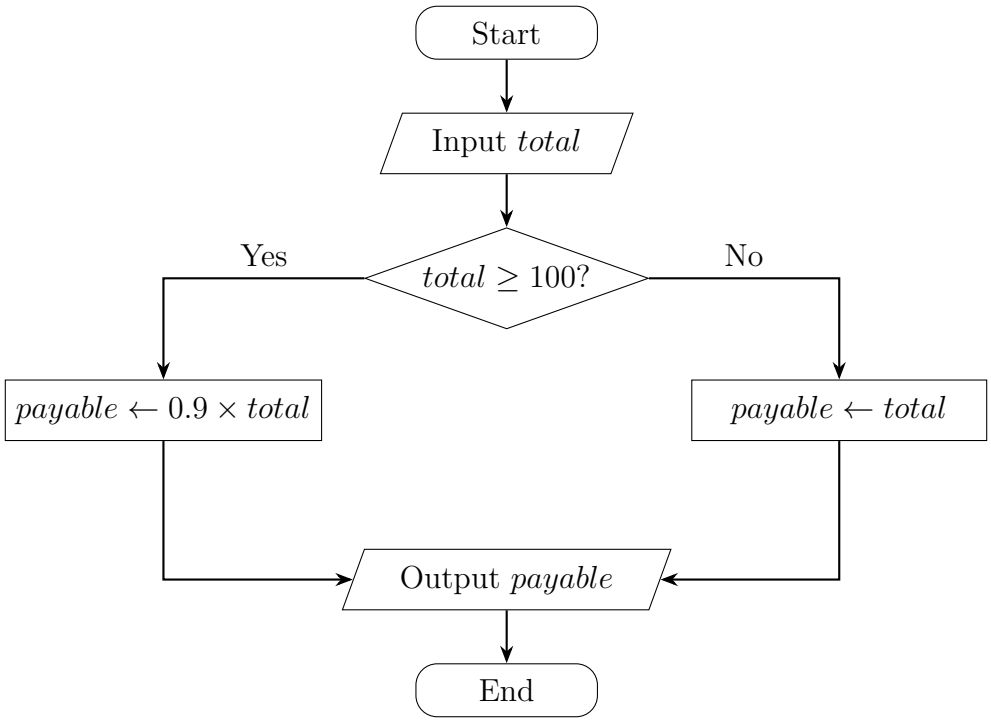
A **flowchart** represents an algorithm's control flow. Read from Start and follow arrowheads, usually downwards. At each decision, evaluate the condition using the *current* variable values, then follow only the matching branch. A loop returns to an earlier step. A trace table records changes in variables, tests and outputs; trace all iterations, including the final failed loop test.

Symbol	Shape	Meaning
Start/End		Rounded rectangle or oval: entry or termination point.
Process/Operation		Rectangle: calculation, assignment or processing step.
Input/Output		Parallelogram: read input or display/write output.
Decision		Diamond: evaluate a condition; label the outgoing branches Yes/No or True/False.
Flowline		Arrow: direction and order of execution, including loop returns.
Connector		Small labelled circle: continue at the matching connector elsewhere on the same page; avoids long or crossing flowlines.

Tracing conventions: $\leftarrow$ means assignment, so $x \leftarrow x + 1$ replaces x with its old value plus one. A comparison such as $x = 5$ inside a decision tests equality. Record outputs only when an output symbol is reached. A variable retains its value until assigned again. Check that every route is defined and that a loop makes progress towards termination.

Example 1 — selection: school shop discount

Input a non-negative purchase total. If it is at least 100 currency units, apply a 10% discount; otherwise keep the original total. Output the amount payable. Assume valid input and use totals for which rounding is unnecessary in this example.



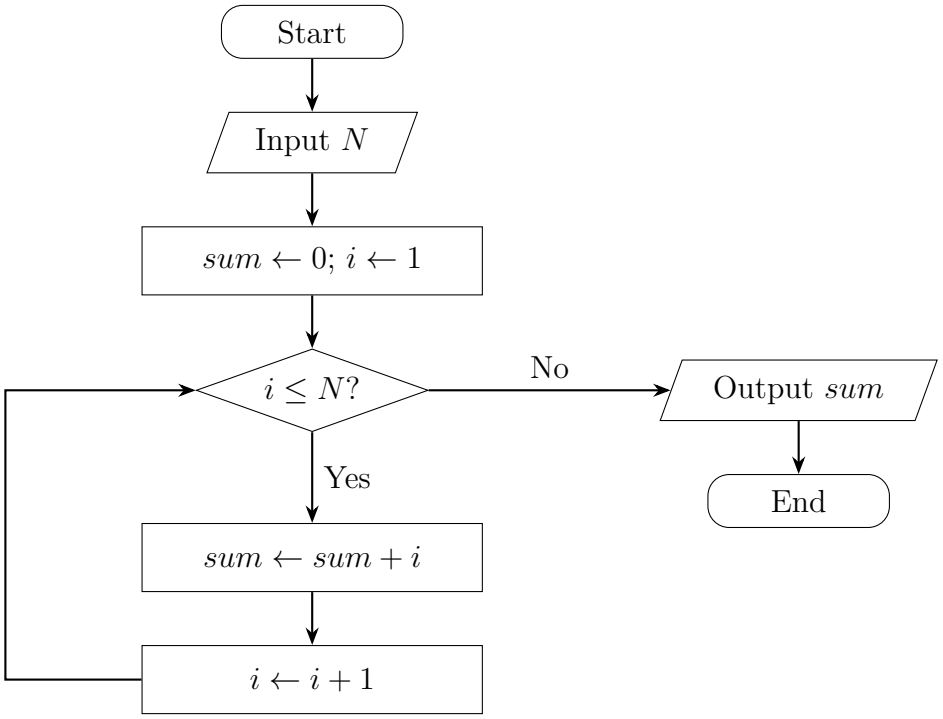
Trace table: independent runs

Input *total*	*total* ≥ 100?	Assigned *payable*	Output
80	False	80	80
100	True	90	90
150	True	135	135

For input 150, execution follows Start → input → decision (Yes) → discounted assignment → output 135 → End. The other assignment is skipped. The boundary input 100 takes the Yes branch because the comparison includes equality.

Example 2 — iteration: sum from 1 to N

Input a non-negative integer N. Add each integer from 1 to N and output the total. Initialize the accumulator before the loop; update both the accumulator and counter on each pass.



Trace for $N = 4$ ("after body" means after both assignments):

Test	i before	sum before	$i \leq 4$?	sum after body	i after body	Output
1	1	0	True	1	2	—
2	2	1	True	3	3	—
3	3	3	True	6	4	—
4	4	6	True	10	5	—
5	5	10	False	—	—	10

There are **four executions of the loop body and five condition tests**. On the final test, $i = 5$, so the No branch outputs 10. For the boundary input $N = 0$, the first test is false and the output is 0. Forgetting to increment i would cause a non-terminating loop for $N \geq 1$; using $i < N$ would omit N from the sum.

IA practice — adapt the room-booking example to a real user

1. Identify an accessible user, such as the teacher responsible for shared rooms. Ask how bookings are currently recorded, what goes wrong and which tasks take time. Record their actual needs rather than assuming the illustrative problem applies to them.

2. Agree a manageable problem specification and measurable success criteria with that user. Replace the example's proposed deadlines, data volumes and usability targets with requirements justified by the user's situation.

3. Explain how the program will transform inputs into outputs, apply all four computational-thinking concepts, and draw a flowchart for a key algorithm such as conflict checking. Explain why automation helps this user and compare it with their current method or an existing booking tool.

4. Plan normal, boundary and invalid test data for each criterion; for example, overlapping bookings, adjacent bookings and an end time before the start. Review a prototype with the user and evaluate the results against the agreed criteria, recording remaining limitations.

This is a planning exercise: a real project needs evidence from the actual user and testing, rather than invented interview findings or claimed results.

B1 completion check: B1.1.1 covers all six specification components and measurable criteria; B1.1.2 defines and illustrates all four CT concepts; B1.1.3 applies them to software development, data analysis, network security, database design and machine learning, as well as the room-booking problem and IA practice; B1.1.4 supplies all six standard symbols and two worked traces covering sequence, selection and iteration.

B2 Programming

B2.1 Programming fundamentals

B2.2 Programming constructs

B2.3 Data structures

B2.4 Programming algorithms

B2.5 File processing

B3 Object-oriented programming (OOP)

B3.1 Fundamentals of OOP for a single class

B3.2 Fundamentals of OOP for multiple classes

B4 Abstract data types (ADTs)

B4.1 Fundamentals of abstract data types

C Appendices

C0.1 My favourite operating system: Arch Linux (+ Hyprland)

C0.1.1 What it is

Arch Linux is a lightweight, rolling-release distribution built around the principle of user-centricity: you start from a minimal base and build up exactly the system you want, installing only the packages you actually need. On top of it I run Hyprland, a dynamic tiling Wayland compositor, which handles all the window management and visual effects. Together they are not a finished product you install, but a system you assemble and shape yourself.

C0.1.2 Why it is my favourite

- Customizability: nothing is assumed for me. Every package, service, kernel module and keybind is a deliberate choice, so the OS fits my workflow instead of the other way around. Hyprland is configured entirely through a plain-text file, so layouts, animations and rules can be tuned down to the pixel.
- Responsiveness and performance: the system ships with no bundled bloatware or background services, so boot times are short and idle resource usage is very low. On the same hardware, a minimal Arch + Hyprland setup feels noticeably snappier than a stock desktop environment.
- Rolling release: packages are updated continuously rather than frozen until the next major version, so I always have current kernels, drivers and toolchains without a disruptive reinstall.
- The AUR and community: the Arch User Repository gives access to an enormous range of community-maintained packages, and the Arch Wiki is arguably the best-documented Linux resource available, which makes troubleshooting a learning exercise rather than a dead end.
- Keyboard-driven workflow: as a tiling compositor, Hyprland automatically arranges windows with no manual dragging, which keeps my hands on the keyboard, reduces context switching and makes multitasking far faster.
- Learning and control: because I built the system, I understand how each layer fits together. Using Arch teaches real operating-system concepts — package management, the boot process, display servers, resource allocation — which connects directly back to the OS material in Section A.

A rolling-release, build-it-yourself system does come with trade-offs: upgrades occasionally break things and demand manual intervention, and there is no vendor support to fall back on. For me that trade-off is worth it — the control and understanding I gain outweigh the convenience I give up.